ORBIT: One-Shot Re-Delegatable Signatures for Scalable Off-Chain Payments

Abstract

Blockchain systems face fundamental scalability challenges: as network size grows, limitations in transaction throughput, privacy, and latency become increasingly pronounced. Off-chain solutions—such as payment and state channels, rollups—alleviate some of these issues but often require complex dispute resolution protocols involving additional assumptions to tackle rollback attacks and also have poor on-chain privacy.

This work introduces one-shot re-delegatable signatures (ORBIT), a novel cryptographic primitive designed to enable secure, scalable off-chain payments by treating signing rights as a form of e-cash. The core feature of ORBIT is *one-timeness*: a bank can delegate signing rights to a user, who can either use it to redeem funds to themselves or re-delegate (or transfer) them to another user exactly once—no more, no less. This one-time transferability prevents double-spending and mitigates rollback attacks without increasing on-chain overhead. Furthermore, ORBIT achieves *history-freeness*: signatures obtained after (re-)delegation are indistinguishable from fresh ones, preserving the privacy of the payment history.

We construct ORBIT using cryptographic assumptions supported by commodity hardware, specifically leveraging *limited-count keys*, i.e., hardware-backed keys that become inaccessible after a predefined number of uses. We detail the ORBIT construction and rigorously analyze its security under realistic adversarial models. In doing so, ORBIT offers a lightweight yet robust alternative to existing off-chain payment mechanisms. Our experimental evaluation demonstrates its strong potential for enabling practical, secure, and private blockchain scalability.

CCS Concepts

• **Security and privacy → Digital signatures**; **Privacy-preserving protocols**.

Keywords

Signatures, layer-2 payments, Blockchain, scalability

ACM Reference Format:

. 2025. ORBIT: One-Shot Re-Delegatable Signatures for Scalable Off-Chain Payments. In *Proceedings of ACM Conference on Computer and Communications Security (Anonymous Submission to ACM CCS 2025).* ACM, New York, NY, USA, 17 pages. https://doi.org/XXXXXXX.XXXXXXX

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Anonymous Submission to ACM CCS 2025,

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Blockchains provide a decentralized platform for users to transact without intermediaries. A fundamental principle of decentralization is requiring the entire network to validate, enforce, and store transactions. While this minimizes trust assumptions and enables secure transactions, its efficiency declines as network size increases. For example, Bitcoin processes about 7 transactions per second (TPS) and Ethereum 15–30 TPS, far below traditional payment networks like Visa, which can handle over 65,000 TPS at peak capacity [18, 24, 30]. As adoption grows, these limitations lead to congestion, high fees, and slow confirmations. Furthermore, scalability constraints introduce inefficiencies such as transaction reordering or maximal extractable value (MEV) attacks, where participants manipulate execution order for profit, undermining fair and predictable execution [13, 35]. Addressing scalability is therefore crucial for enhancing usability and maintaining stable economic incentives.

A vast body of research (see surveys [1, 2, 31, 34]) explores scalability solutions, broadly categorized into:(1) *layer 1* solutions, which enhance the consensus layer and blockchain infrastructure. Examples include Ethereum 2.0's sharding [17], Near Protocol's Nightshade [33], and Polkadot's parachains [44]. And (2) *layer 2* or *off-chain* solutions, which reduce on-chain computation, and storage overhead by shifting transaction processing off-chain while committing only a succinct state on the chain. Examples include payment channels [26, 32, 40, 41], state channels [29], and rollups [19, 39]. These solutions also improve privacy by minimizing the information recorded on-chain about individual payments, reducing the exposure of transaction details while preserving correctness.

Layer-2 solutions have gained particular traction as they *do not* require extensive changes to the blockchain infrastructure, making them easier to deploy compared to layer-1 solutions. However, a key challenge remains: *rollback attacks*. Since not all transactions are immediately recorded on-chain, malicious users may revert to an outdated state that benefits them. Current solutions address this with intricate payment mechanisms involving recovery transactions, strict online and synchrony assumptions for both parties [4], and economic deterrents such as transaction fees or collateral [11, 42]. These countermeasures introduce complexity and operational overhead [5], limiting the efficiency and practicality of layer-2 systems.

An alternative approach was proposed by Amos et al. [3], leveraging quantum *one-shot signatures* to enable off-chain payments without rollback attacks. Their technique, based on the no-cloning principle of quantum mechanics, ensures that payment states cannot be duplicated or outdated payments reinstated. Unlike payment channels, this solution does not require on-chain *"per-parties" setup*, where senders and receivers must pre-register on-chain through unique setup transactions. Furthermore, it eliminates the need for complex payment mechanisms and the continuous involvement of external entities for transaction finality. While this approach offers some benefits, it fundamentally relies on signing tokens in quantum superposition states, making it incompatible with classical

blockchain infrastructure. Moreover, it lacks payment privacy, as a coin's transaction history remains fully traceable. This raises a fundamental question:

Can we design a cryptographic solution that supports layer-2 payments that (1) allows users to transact on the fly, (2) remains compatible with existing blockchain systems, (3) is privacy-preserving, and (4) resists rollback attacks without relying on complex and costly dispute resolution mechanisms?

1.1 Our Contribution

In this work, we answer this question by exploring a new cryptographic primitive inspired by quantum one-shot signatures but designed for the classical setting. Our approach leverages security assumptions on widely deployed hardware components, found in billions of smartphones today. Our contribution in this work can be summarized as follows:

- We introduce *One-shot Re-delegataBle sIgnaTures* (ORBIT), a new cryptographic primitive designed to work with standard digital signature schemes. In ORBIT, a third-party delegates it's signing authority by issuing signing tokens to users for free in an *offline phase*. However, these tokens remain *dormant* and useless until later, during the online phase, where a user can *activate* their token with the help of the bank. Once activated, the token can be used in two ways:
  (1) *Re-delegation*: The user can delegate signing to another user (the recipient), and activating the recipient's signing token.
  (2) *Delegated signing*: The user can sign a message.
  A key feature of ORBIT is its *one-timeness*–each activated token can be used exactly once. Additionally, ORBIT ensures *history-freeness* (signatures from delegation are indistinguishable from fresh signatures under the same public key) and *verifiability* (both pre- and post-activation tokens are verifiable to recipients).

  Off-Chain Payments Using ORBIT**.** Our motivation for designing ORBIT stems from the challenge of rollback attacks in layer-2 payments. Using ORBIT, any user can obtain signing tokens from a bank-like entity at any time, without requiring pre-registration. To make a payment, a user, Alice, first activates her signing token with assistance from the bank. In return, Alice locks 1 coin in a public smart contract setup by the bank. To pay 1 coin to Bob, Alice re-delegates her signing token to Bob via an off-chain transaction, consuming her activated token while simultaneously activating Bob's token. Bob can then either continue the chain by re-delegating the token to another user or sign a special message m to generate a signature σ. When Bob posts (m, σ) to the smart contract, he is able to redeem 1 coin locked in the smart contract.
  An astute reader may notice similarities between this mechanism and the *burn-and-prove* mechanism used in Zerocoin [28], where a spent coin is "burned" and a new one is "minted" while preserving anonymity. However, in Zerocoin, double-spending must be manually detected and prevented, since multiple users could attempt to redeem the same coin. In contrast, ORBIT inherently prevents double-spending through its one-timeness property: each activated token can only be used once, ensuring that the transfer is atomic and rollback attacks are impossible.
  Unlike traditional payment channels, our approach eliminates the need for "per-parties" setup, dispute resolution requiring online assumptions, or synchrony assumptions, making it a lightweight and practical layer-2 payment solution. A detailed technical overview is provided in Section 2.
- We present the first efficient construction of ORBIT for the ECDSA signature scheme that satisfies one-timeness and history-freeness. We also propose $\mathsf{ORBIT}_{\mathsf{ZK}}$, a modified version of our construction that uses non-interactive zero-knowledge to additionally ensure verifiability. At a high level, our schemes build on the folklore application of Yao's two-party computation, interpreting tokens as efficiently executable garbled circuits for the signing function. To avoid costly circuit representations (like a boolean circuit from traditional Yao's approach), we instead employ an efficient secret-sharing approach, where signing reduces to reconstructing a secret from encrypted shares tied to input bits. Yeoh et al. [45] previously demonstrated this approach for ECDSA. We extend it to Waters signatures, enabling re-delegation: Waters signatures double as decryption keys in the Waters Identity-based Encryption (IBE) scheme. This allows Alice to either generate a Waters signature and activate Bob's token or evaluate the ECDSA branch and obtain a delegated ECDSA signature. Consequently, we also obtain ORBIT for Waters signature, where the ECDSA branch can be replaced with a second Waters branch when the signer's keys are Waters keys. To ensure that only one label per input bit is ever decrypted, we employ the n-use key exchange (nUKE) primitive [45]. This primitive can be instantiated under the lightweight and practical *limited-count keys* hardware assumption—where hardware-backed keys become inaccessible after a predefined number of uses. This feature is supported by commodity hardware, including Android smartphones (via the Keystore API) and leading hardware security modules such as Nitrokey HSM[1] and Thales Luna HSM[2].
- Finally, we contribute a prototype implementation of ORBIT, realized on a Google Pixel Pro smartphone using the Android Keystore system. The redelegation in the ORBIT system only takes about 300 ms and the final signature redemption takes around 1.5 seconds. The redelegatable pre-signature size ranges from 8 KB to 776 KB depending on the configuration.

1.2 Related Work

Teechain [26] was a scalability improvement proposal that relied on users' trusted hardware to mitigate rollback attacks. However, their approach has three main drawbacks: (1) They rely on *trusted execution environment (TEE)*, which imposes a significantly stronger hardware assumption than the *limited-count keys* required for constructing ORBIT. In contrast, our assumption is lightweight and practical: It does not require secure code execution but only a monotonic counter mechanism, a feature already supported by many hardware systems such as TPMs and HSMs. (2) They required per-parties setup on the chain like payment channels, i.e., the sender and the receiver of a payment to set up on-chain channels. (3) Finally, both parties were required to be continuously online during

[1] https://shop.nitrokey.com/shop/nkhs2-nitrokey-hsm-2-7
[2] https://thalesdocs.com/gphsm/luna/7/docs/network/Content/sdk/design/operation_count.htm

payment, unlike our ORBIT-based payment solution, where the recipient can go offline during the payment process.

Ekiden [10] also proposed combining blockchains with trusted hardware, focusing on offloading contract execution to TEEs for better scalability and confidentiality. While Ekiden achieves scalability benefits, it inherits the strong trust assumption on TEEs, and its design primarily targets general-purpose smart contract execution rather than streamlined, offline-capable payment solutions as in ORBIT. FastKitten [14] proposed another TEE-based approach to speed up payment channels by enabling efficient dispute resolution. However, like Teechain, FastKitten relies on TEEs for security and still requires a certain level of synchrony between parties during the execution. In contrast, ORBIT allows secure asynchronous payments using a more reasonable hardware assumption.

2 Technical Overview

In this section, we will present a high level technical overview of the design of ORBIT and our construction of ORBIT. The fundamental motivation behind ORBIT is to facilitate off-chain payments between users of a blockchain system as stated in Section 1.1. Therefore, before we begin the overview, it will be useful to summarize the key goals of the system:

- *No registration:* Users can transact without on-chain enrollment.
- *Legacy-friendly:* Easily deployable in existing blockchains.
- *Fungibility:* Payments are history-free and unlinkable.
- *Rollback protection:* Secure without on-chain dispute resolution.

2.1 Straw-man Approach

As a starting point, consider one-shot signatures [3], which allow a user to sign exactly one message. A basic off-chain payment mechanism can be imagined using this primitive: a trusted entity (e.g., a bank) issues certified public keys by signing them in exchange for cryptocurrency. These keys serve as one-shot verification keys.

To initiate a payment, Alice signs Bob's public key using her one-shot signing token, creating a verifiable re-delegation. Since the token is single-use, Alice cannot sign any other message, ensuring double-spend resistance. Bob can either re-delegate the token to another user by signing their key or redeem the cryptocurrency by signing a designated message presented by the bank.

While simple, this straw-man approach as discussed before, has two major limitations: (1) *Lack of privacy*: The full transaction path (bank's signature on Alice's key, Alice's signature on Bob's key, and Bob's final redemption signature) is publicly visible, allowing the bank to reconstruct the entire payment chain and link users; (2) *Quantum dependence*: Current constructions of one-shot signatures require quantum capabilities (e.g., tokens in superposition), rendering them incompatible with classical blockchain infrastructures.

An alternative is to use one-time delegatable signatures, where a signer can transfer signing rights once while enforcing a single-use constraint. However, such primitives cannot be constructed under standard assumptions: a malicious user could always copy a delegation and reuse it to sign multiple messages. Krenn and Lorünser [25], proposed using smart contracts to enforce single-use. However, their solution requires verifiers to access the contract state to validate signatures, resulting in non-standard signatures and complicating verification. More recently, Yeoh, Hanzlik, and Valta [45] addressed this by relying on a weak hardware assumption supported by modern Android devices: that hardware-protected keys can be configured for a limited number of uses. Leveraging this, they constructed a one-time delegatable signature scheme where the final output is a standard ECDSA signature.

Unfortunately, directly replacing one-shot signatures with one-time delegatable signatures in the above payment system introduces new challenges. In the one-shot setting, users generated their own keys, and the public key effectively served as an address—similar to existing cryptocurrencies. Payments were made by signing another user's key, forming a simple transfer chain. In contrast, one-time delegatable signatures focus on delegating signing capability itself rather than signing a user's key.

Thus, applying one-time delegatable signatures would require the bank to delegate its signing authority upfront to users, risking misuse: users could forge signatures on arbitrary documents or interact outside the intended payment flow. Moreover, assigning signing capability only to Alice (and not Bob) would not suffice, since these primitives are inherently non-redelegatable—Bob would have no way to continue the chain. In short, while one-time delegation enables single-use signatures, it lacks the necessary control and reusability properties needed for a secure and privacy-preserving off-chain payment system.

2.2 Defining ORBIT: One-shot Re-Delegatable Signatures

To address the limitations of the strawman system, we introduce a new cryptographic primitive called *one-shot re-delegatable signatures* (ORBIT), which we define with respect to a standard digital signature scheme. To achieve re-delegation compared to prior work [45] that only give single-use delegatable signatures, we introduce a key innovation: splitting the delegation process into an offline and an online phase. Intuitively, the offline phase is a "free of cost" phase where users obtain delegation rights (on paper), without the power to exercise these rights. Later, during the online phase the users earn the power to execute their delegated rights to sign.

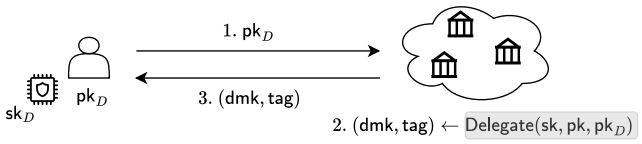


Figure 1: Offline Phase: Dormant Key Issuance

Offline Phase (Fig. 1). In this phase, using the Delegate(sk, pk, pk_D) algorithm, the signer entity (the bank) generates dormant keys dmk, which are bound to a tag tag. Here, sk is the entity's secret key for the re-delegatable signature scheme, and *pk* is the corresponding public key pk. The user's public key pk_D and secret key sk_D are generated using DKeyGen(λ). A key advantage of this approach is that the bank can issue dormant keys independently of any payments, as these keys remain useless unless activated by the bank. This mechanism functions similarly to pre-issued blank checks that can be activated when needed. Here, we assume that the bank is honest-but-curious unless ORBIT supports verifiability, in which case the bank can be considered malicious.

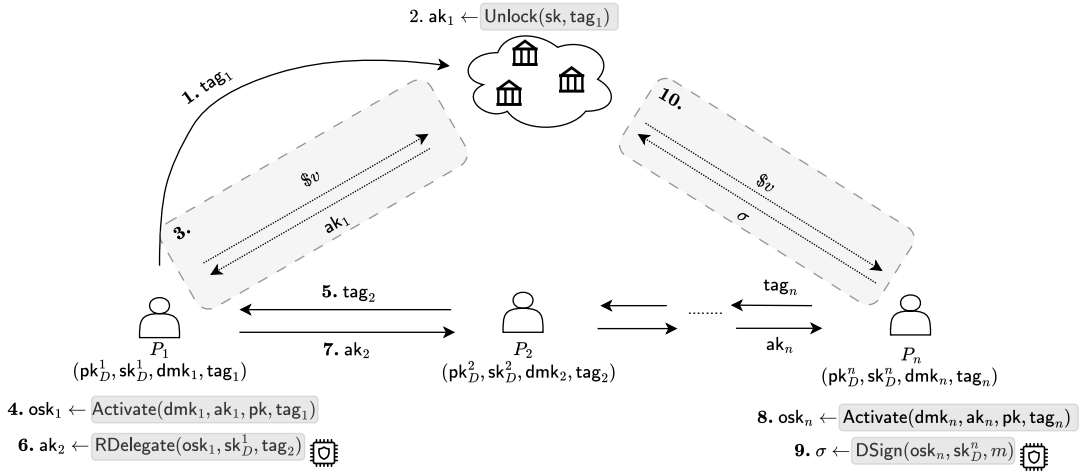


Figure 2: The online Phase of ORBIT desribed as a protocol flow. In the payment protocol, the exchange in step 3 and step 10 involve an atomic data exchange between the banks and the user.

Online Phase (Fig. 2). A dormant key is activated by running Activate(dmk, ak, pk, tag), using an active key ak_1 associated with the same tag tag_1 and the re-delegatable scheme public key pk, resulting in the one-shot signing key osk_1 (step 4). Before activation, the bank unlocks Alice's dormant key (identified by tag_1) by running $\mathsf{Unlock}(\mathsf{sk}, \mathsf{tag}_1)$ (step 2) and providing Alice with the active key ak_1 in exchange for a payment ($\$v$)(step 3).

At this point, Alice can either re-delegate the signing capability or sign on behalf of the bank. In the former case, Alice runs $\mathsf{RDelegate}(\mathsf{osk}_1, \mathsf{sk}_D^1, \mathsf{tag}_2)$ using her one-shot signing key osk_1 and a new tag tag_2 associated with Bob's dormant key (step 6). This grants Bob the active key ak_2, allowing him to activate his dormant key. Bob and any subsequent recipients can repeat this process. A crucial property of this system is that each party can re-delegate only once, as it requires using a one-shot signing key. The final recipient in the chain receives the last one-shot signing key osk_n and uses $\mathsf{DSign}(\mathsf{osk}_n, \mathsf{sk}_D^n, m)$ (step 9) to generate a signature which can then be exchanged with the bank for $\$v$ coins (step 10).

In addition to the syntax described above, ORBIT must satisfy two key security properties: *one-timeness* and *history-freeness*.

- *One-timeness* strengthens the standard notion of unforgeability. In traditional signature schemes, unforgeability ensures that an adversary cannot produce a valid signature unless they have seen one from the signer. However, since our scheme allows delegation, we need a stronger guarantee: even if the signer delegates signing rights, an adversary should only be able to produce as many valid signatures as the number of activation keys it has received, plus any direct signatures from the signer.
  Formally, this is similar to the *one-more unforgeability* notion used for blind signatures. In the security experiment, the adversary can request as many dormant keys and plain signatures from the signer as it wants. It can also request activation keys for chosen tags tag. To win, the adversary must produce $l + 1$ valid message-signature pairs, where each message is distinct from any previously queried to the signer. Crucially, the adversary is limited to making at most l queries to the activation oracle.
- *History-freeness* ensures that signatures produced by delegated signing are indistinguishable from those generated directly by the signer. In other words, a third party observing a signature cannot tell whether it came from the original signer or by delegation. Note that the signer can always link a final signature to the dormant key originally issued to a user.
  Formally, this is modeled through an experiment where the adversary receives a tag tag and an honestly generated public key pk_D. The adversary can create an arbitrary delegation chain by interacting with oracles that provide dormant and activation keys, as well as oracles that generate non-delegated signatures. Once the adversary outputs a valid activation key for the challenger tag tag, it is presented with either a non-delegated signature or a signature produced by DSign with the one-shot secret key for tag, depending on a hidden bit b. The adversary's goal is to guess b correctly. A crucial aspect of history-freeness is that no delegation history is embedded in the signature. Furthermore, since the adversary never obtains the signing key sk, it does not need to hold with respect to the signer, as previously noted.

We may require ORBIT to be used in applications where the signer is malicious when creating the dormant keys. Therefore, we introduce an additional property for ORBIT described below:

- *Verifiability* ensures that a valid dormant key can be activated into a valid one-shot signing key osk. This property is particularly important because dormant keys are issued in the offline phase. By allowing users to verify their validity beforehand, the verification function protects them from receiving invalid dormant keys that cannot be activated later. To achieve this, we assume that there exists a verification function VerifyD that takes as input a dormant key dmk with the corresponding tag tag, the signer's public key pk, and the user's public key pk_D. The function outputs 1 if the dormant key is valid for the given inputs.

2.3 Flying into ORBIT efficiently

With the above definitional framework, we set out to construct our ORBIT scheme, using the single-use delegatable signature scheme proposed by Yeoh, Hanzlik, and Valta [45] as the starting point.

Their scheme is inspired by Yao's two-party computation (2PC) construction and can be informally described as an efficient garbling circuit for ECDSA signatures, coupled with hardware assumptions to enable a non-interactive oblivious transfer (OT). The high-level idea is that the delegator gives the delegatee a garbled circuit that can be evaluated once to produce an ECDSA signature on a message (of the delegatee's choice) under the delegator's public key.

To explain the underlying mechanism more precisely, recall the structure of an ECDSA signature ($r = g^k \mod q, s = k^{-1}(r \cdot sk_{\mathsf{ECDSA}} + \mathsf{H}(m))$). Here, g is the group generator for a group of order q, m is the message, $\mathsf{sk}_{\mathsf{ECDSA}}$ is the signing key, k is the signing randomness, and H is a hash function. The key idea of efficient garbling is to prepare shares of the signature component s based on the bits of $\mathsf{H}(m)$, denoted by $h_1, \ldots, h_{\ell_m} \in \{0,1\}$. The authors observe that s can be rewritten as:

$$s = k^{-1}(r \cdot sk_{\mathsf{ECDSA}}) + \sum_{i=1}^{\ell_m} 2^{i-1} \cdot h_i \cdot k^{-1}.$$

Assuming payloads are defined as $p_0 = k^{-1}(r \cdot sk_{\mathsf{ECDSA}})$, $p_{i,0} = 0$, and $p_{i,1} = 2^{i-1} \cdot k^{-1}$, then for a given message hash with bits $h_1, \ldots, h_{\ell_m}$, one can reconstruct s as: $s = p_0 + \sum_{i=1}^{\ell_m} p_{i,h_i}$. However, revealing this representation highlights a vulnerability: having access to p_0 and any $p_{i,1}$ allows an adversary to recover the signing key sk_{ECDSA}. To mitigate this, the scheme introduces an additive secret sharing of 0, i.e., values $t_0, \ldots, t_{\ell_m}$ such that $t_0 + \ldots + t_{\ell_m} = 0$. With this sharing, the payloads become: $p_0 = k^{-1}(r \cdot sk_{\mathsf{ECDSA}}) + t_0$, $p_{i,0} = t_i$, and $p_{i,1} = 2^{i-1} \cdot k^{-1} + t_i$. Using an OT protocol, the delegator can ensure the delegatee receives only one of $p_{i,0}$ or $p_{i,1}$ per index i (depending on the value of h_i), thus maintaining the secrecy of the signing key. This setup resembles Yao's 2PC: p_0, r and encryptions of all $p_{i,0}$ or $p_{i,1}$ are the garbled circuit, while the labels are decryption keys to decrypt only one of $p_{i,0}$ or $p_{i,1}$ per position i.

However, standard OT requires interaction during the signing process, which is undesirable. To overcome this, Yeoh et al. introduce a novel cryptographic primitive called *n-use key exchange* (nUKE). This mechanism extends classical key exchange with a restriction: a user can attach an attestation σ_{att} to its public key pk_N, limiting its use to n derivations. Thus, the user can compute exactly n shared secrets with different parties, which can be used to encrypt the payloads $p_{i,0}$ and $p_{i,1}$, i.e., for each position i the delegator can pick two unique keys $\mathsf{pk}_{i,0}$ and $\mathsf{pk}_{i,1}$, where the secret between pk_N and $\mathsf{pk}_{i,b}$ is used to decrypt payload $p_{i,b}$ for $b \in \{0,1\}$. In other words, the process can be seen as a non-interactive OT where the delegatee receives $2 \cdot \ell_m$ ciphertexts and is only able to decrypt $n = \ell_m$ of them, each for one position $i \in [\ell_m]$.

While this solves the interaction problem, it still doesn't entirely prevent misuse. The authors employ an all-or-nothing approach to strengthen the construction: the delegatee must choose precisely one payload to decrypt per index $i \in [\ell_m]$. Attempting to decrypt both $p_{i,0}$ and $p_{i,1}$ for any index i results in failure to decrypt *any* payloads. This is achieved through a second layer of *additive secret sharing*: the delegatee must collect *one share per index*, and only when *all* shares are present can the actual secret be reconstructed.

Despite this, the scheme in [45] lacks two features necessary for our purposes: support for re-delegation and the concept of dormant keys. In their scheme, the delegatee immediately receives a one-shot signing key without any need for an activation key.

Re-delegation using Waters Signatures. Our idea to enable re-delegation in ORBIT is to use signatures that can also serve as witnesses or decryption keys for ciphertexts. In other words, by leveraging the connection between signatures and identity-based encryption (IBE) keys—recently explored [15, 27] in the context of efficient witness encryption schemes—we can tightly bind delegation capabilities to signature possession.

Guided by this intuition, we first observe that a garbling technique similar to that in [45] can be applied to Waters signatures [23, 43]. A Waters signature has the form

$$(\sigma_1, \sigma_2, \sigma_3) = (g_1^{\alpha\beta} \cdot (u_0 \prod_{i=1}^{\ell} u_i^{m_i})^r, g_1^r, g_2^r),$$

where g_1, g_2 are group generators of groups $\mathbb{G}_1, \mathbb{G}_2$, respectively, and only σ_1 depends on the message bits $m_1, \ldots, m_\ell$. Analogous to the ECDSA case, we can express σ_1 as: $p_0 \cdot \prod_{i=1}^{\ell} p_{i,m_i}$, with $p_0 = g_1^{\alpha\beta} \cdot u_0^r, p_{i,0} = 1_{\mathbb{G}_1}, p_{i,1} = u_i^r$. However, this again risks revealing the signing key $g_1^{\alpha\beta}$. To prevent this, we apply a multiplicative secret sharing of $1_{\mathbb{G}_1}$, using shares $t'_0, \ldots, t'_\ell$. All other steps mirror the ECDSA construction. At this point we have a one-shot delegatable signature scheme for Waters signatures.

Furthermore, our construction allows both ECDSA and Waters branches to coexist, allowing the delegatee to choose which signature type to obtain. However, this choice is bound to the delegatee's nUKE key, ensuring only one signature type is derived. This combined approach forms the one-shot signing key osk in ORBIT. As we explain later, the choice of signature type reflects the intended action: ECDSA for signing and Waters for re-delegation.

A key observation is that Waters signatures are equivalent to identity-based decryption keys in the Waters IBE scheme [43]. In other words, possessing a Waters signature for message m grants decryption capabilities for identity m. We leverage this in ORBIT: we define a special secret r_{tag} encrypted under identity = tag via Waters IBE. This secret is required to reconstruct a component a_{tag} used in deriving the key that decrypts the garbled payloads. Without knowledge of r_{tag}, the delegatee cannot recover the payloads and complete the signature.

In our scheme, the Waters signature serves as the *activation key* ak, while the dormant key dmk includes Waters IBE encryption of r_{tag} under the identity tag. This structure ensures that even with access to a one-shot signing key as above, the delegatee cannot sign unless it possesses a valid Waters signature under tag.

Thus, evaluating the efficient garbled circuit, as described here and in [45], requires more than just executing the nUKE exchange and decrypting the payloads. The delegatee must also retrieve r_{tag}, which is only possible if it holds a Waters signature under tag. This can be obtained directly from the delegator (step 3 in Figure 2) or through re-delegation. In the latter case, another user executes the efficient garbled circuit to produce a Waters signature under tag for the current user. This process continues until one user chooses to sign (using the ECDSA branch, step 9 of Figure 2), at which point an ECDSA signature is produced. Section 5.1 provides further details about the full protocol and construction of ORBIT.

2.4 From Strawman to Robust Payment System

One might assume that a straightforward application of the one-shot re-delegate signatures, in the previously described straw-man payment system Section 2.1, would be sufficient for a layer-2 scalable payment solutions. However, this assumption only holds under the condition that the bank is semi-honest and does not act maliciously—a rather strong assumption in blockchain ecosystems given the potential financial incentives for dishonest behavior. For instance, the bank could exploit the system in the following ways.

Fair exchange solutions. In step 3 of the online phase (see Figure 2), the bank could receive payment without issuing the corresponding activation key. To address this, we employ a *fair exchange* protocol that ensures cryptocurrency from the user is transferred to the bank in an atomic manner for a Waters signature of the bank on a designated message. Notably, in our construction, the activation key takes the form of such a signature. This protocol can be instantiated generically using simple smart contracts using the techniques from [38] or by adapting the approach from [22], where cryptocurrency is exchanged for BLS signatures.

A fair exchange is also required in step 10 of Figure 2, where the user exchanges an ECDSA signature on the blockchain for a payment from the bank. To mitigate the risk of the bank issuing valid activation keys but later withholding payment for the ECDSA signatures, we can use the same smart contract as above, that compensates the holder of an ECDSA signature on a fresh (i.e., not previously redeemed) message. This is facilitated through the atomic swap algorithm in step 3, which transfers a portion of the cryptocurrency to the smart contract, with part of this amount taken by the bank as an application-specific fee.

Distributed bank. Yet another malicious strategy would involve the bank leveraging its knowledge of the signing key. Specifically, it could execute the contract independently and defraud honest clients by producing ECDSA signatures to redeem coins from the above contract. To counter this, we adopt a generic approach through distributed trust, where we decentralize the bank and execute the delegation process in parallel. In step 10, the client must present n ECDSA signatures on the same message, verifiable under the public keys of n different banks—where n is an application-specific parameter. With this strategy, as long as at least one bank remains semi-honest, only clients can successfully redeem funds via the smart contract, and no coalition of malicious banks can independently extract currency.

Multiple denominations. Interestingly, this approach also opens the door to supporting multiple denominations within the payment protocol. Our current solution supports only a single denomination, i.e., a fixed amount $\$v$ coins. However, using multiple public keys allows us to represent different values—for instance, pk_1 could correspond to \$1 coin, while pk_5 denotes \$5 coins. We leave the design of a more efficient mechanism for distributing banks and supporting coin denominations to future work. In this paper, we focus on the core component of our system: ORBIT.

Cross-chain payments. Our layer-2 payment mechanism interacts with the blockchain only twice—during the exchanges in step 3 and step 10 of Figure 2—each requiring a single transaction on the chain. All intermediate payments occur off-chain and are independent of any blockchain, allowing users across different systems to transact seamlessly. Furthermore, step 3 and step 10 can be executed on different blockchains, say C_a and C_b, provided sufficient balance exists in the contract on C_b for users to redeem coins. A detailed exploration of this cross-chain capability is left to future work, as it promises significant advances in layer-2 interoperability.

2.5 Discussion on Hardware assumption

A central pillar—and indeed the glue—of our ORBIT construction is the limited-count key assumption: the notion that a hardware-backed cryptographic key can only be used a predetermined number of times. Unlike hardware assumptions made by many recent works that rely on trusted execution environments (TEEs), the limited-count key assumption offers a lightweight, practical alternative.

TEEs like Intel SGX are commonly used to provide confidentiality and integrity for code running on untrusted machines. They enable the emulation of a trusted third party or enforce honest behavior from potentially malicious actors. However, this comes with significant complexity and a wide attack surface. TEEs are notoriously vulnerable to side-channel attacks, which can undermine both confidentiality and integrity.

In contrast, the limited-count key assumption requires significantly less trust and is easier to implement. Consider a hardware component that securely stores cryptographic keys. To enforce limited use, the only additional requirement is a monotonic counter bound to each key. This counter increments with every use and is checked before the key can be accessed, an access policy with minimal overhead. This idea has been explored in the context of Trusted Platform Module (TPMs) by Sarmenta et al. [36].

Crucially, the fundamental assumption, that hardware-backed keys cannot be extracted from secure memory, is already widely trusted and deployed. Hardware Security Modules (HSMs) protect PKI keys on servers, Smart cards can secure credentials in payment systems and biometric passports, and Fast IDentity Online (FIDO) keys are safely stored in secure enclaves and hardware authenticators. Moreover, TPMs are already trusted to manage critical cryptographic material. Adding a simple counter-check before key usage introduces negligible additional risk.

We argue that this assumption is *lightweight* precisely because it is based on existing and well-established trust in the secure storage of cryptographic keys. Unlike TEEs, which aim to protect arbitrary computation with confidentiality and integrity guarantees, limited-count keys provide a narrowly scoped, targeted capability. And that is their strength: the assumption does not demand more trust than what is already required for key storage—a trust that underpins many real-world systems today.

Multi-device policy. To reduce reliance on a single hardware manufacturer, the delegator can enforce a policy requiring the delegatee to utilize multiple devices during the delegation process. This mitigates the risk associated with trusting a single manufacturer. Importantly, the hardware assumption remains the only line of defense against a malicious delegatee attempting to exploit the one-shot signing key and recover the delegator's signing key. We now demonstrate how this multi-device policy can be efficiently incorporated into our scheme from Section 5.1.

Assume the delegatee provides public keys $\mathsf{pk}_{N,1}, \ldots, \mathsf{pk}_{N,k}$, each accompanied by attestations from different manufacturers

confirming that these are limited-count keys. We modify the computation of $c_{i,0}$ and $c_{i,1}$ in the Delegate algorithm (see Fig. 4). Rather than relying solely on the keys $K_{i,0}$ and $K_{i,1}$, we compute the following keys for each $i \in [\ell_m]$ and $j \in [k]$:

$$K_{i,1,k} \leftarrow \mathsf{nUKE.Shared}(\mathsf{sk}_{i,1,k}, \mathsf{pk}_{N,k}),$$
$$K_{i,0,k} \leftarrow \mathsf{nUKE.Shared}(\mathsf{sk}_{i,0,k}, \mathsf{pk}_{N,k}).$$

These keys are then used to compute the updated $c_{i,0}$ and $c_{i,1}$:

$$c_{i,0} := \mathsf{H}_2(K_{i,0,1}, \ldots, K_{i,0,k}) \oplus (a_i || b_{i,0}),$$
$$c_{i,1} := \mathsf{H}_2(K_{i,1,1}, \ldots, K_{i,2,k}) \oplus (a_i || b_{i,1}).$$

The key idea is that decrypting $c_{i,0}$ and $c_{i,1}$ requires all devices, thereby distributing trust across multiple hardware manufacturers.

3 Preliminaries

We denote by $y \leftarrow \mathcal{A}(x)$ the execution of an algorithm $\mathcal{A}$ on a input x and with a output y. By $r \leftarrow\!\$\ S$, we mean that r is chosen uniformly at random over the set S. We will use $1_{\mathbb{G}}$ to denote the identity element in group $\mathbb{G}$ and $[n]$ to denote the set $\{1, \ldots, n\}$. We used $|S|$ to denote the size of the set. By $\mathcal{A}^{O}$, we denote an algorithm $\mathcal{A}$ that has access to an oracle O.

Definition 3.1. *Let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ be three cyclic group of prime order q with generators $g_1 \in \mathbb{G}_1$ and $g_2 \in \mathbb{G}_2$. A **bilinear pairing** is an efficiently computable function $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ satisfying the following properties: (Bilinear:) $\forall (S, T) \in \mathbb{G}_1 \times \mathbb{G}_2, \forall a, b \in \mathbb{Z}_q$, $e(S^a, T^b) = e(S, T)^{ab} = e(S^b, T^a)$. (Non-degenerate:) $e(g_1, g_2) \neq 1_{\mathbb{G}_T}$. We consider Type-3 pairing in this paper as is the most efficient for a relevant security parameter. Commonly used Type-3 pairing friendly curves includes BN [7] and BLS [6] curves.*

Standard cryptographic primitives. We omit a complete description of standard primitives like digital signatures DS = (KeyGen, Sign, Verify) and non-interactive zero-knowledge proofs NIZK = (NIZK.Setup, NIZK.Prove, NIZK.Verify), as well as the standard ECDSA signature scheme. For completeness, the full descriptions can be found in Appendix A. However, we will make the following remark about the security of the ECDSA signature scheme.

Remark 1. *The ECDSA signature scheme is unforgeable even if the adversary can make "raw" signing queries, i.e., query the signing oracle with $\mathsf{H}(m)$ instead of plain m. This result holds in the generic group model, as noted in [21].*

Waters signature scheme. Waters signature scheme [23, 43] is a signature scheme based on the computational Diffie-Hellman (CDH) assumption. The Waters signature scheme is defined as follows:

- $\mathsf{KeyGen}_{\mathsf{Wat}}(\lambda)$: choose a vector $U = (u_0, u_1, ..., u_\ell)$ where each $u_i \leftarrow\!\$\ \mathbb{G}_1$, sample $(\alpha, \beta) \leftarrow\!\$\ \mathbb{Z}_q \times \mathbb{Z}_q$, then set $A := g_1^{\alpha}$, $B := g_2^{\beta}$, and output $(g_1^{\alpha\beta}, U)$ as the signing key and (A, B, U) as the verification key.
- $\mathsf{Sign}_{\mathsf{Wat}}(\mathsf{sk}, m)$: sample $r \leftarrow \mathbb{Z}_q$, parse $(g_1^{\alpha\beta}, (u_0, u_1, ..., u_\ell)) := \mathsf{sk}$, compute $(\sigma_1, \sigma_2, \sigma_3) := (g_1^{\alpha\beta} \cdot (u_0 \prod_{i=1}^{\ell} u_i^{m_i})^r, g_1^r, g_2^r)$ where $m_1, m_2, \ldots, m_\ell$ is the bit encoding of the message m, and output a signature $\sigma := (\sigma_1, \sigma_2, \sigma_3)$.
- $\mathsf{Verify}_{\mathsf{Wat}}(\mathsf{vk}, \sigma, \mathsf{m})$: parse $(\sigma_1, \sigma_2, \sigma_3) := \sigma$, and output 1 if and only if $e(A, B) \cdot e(u_0 \prod_{i=1}^{\ell} u_i^{m_i}, \sigma_3) = e(\sigma_1, g_2)) \wedge e(\sigma_2, g_2) = e(g_1, \sigma_3)$.

Identity-Based Encryption. Identity-Based Encryption (IBE) is a type of public-key encryption where a user's public key is derived from a user's identifier. IBE can be generically transformed into a signature scheme using Naor's transform [9, 12] or be instantiated into a signature scheme with some modification. Waters IBE shares similarities with Waters signature whereby the decryption key and signature are the same. The security of Waters IBE is based on decisional bilinear Diffie-Hellman (DBDH). IBE is a tuple of algorithms IBE := (Setup, KeyDer, Encrypt, Decrypt) and we describe Waters IBE using some algorithms from Waters signature as follows:

- $\mathsf{Setup}_{\mathsf{Wat}}(\lambda)$: execute $(\mathsf{sk}, \mathsf{vk}) \leftarrow \mathsf{KeyGen}_{\mathsf{Wat}}(\lambda)$, then output a secret key sk and a public key $\mathsf{pk} := \mathsf{vk}$.
- $\mathsf{KeyDer}_{\mathsf{Wat}}(\mathsf{sk}, \mathsf{ID})$: execute $(\sigma_1, \sigma_2, \sigma_3) \leftarrow \mathsf{Sign}_{\mathsf{Wat}}(\mathsf{sk}, \mathsf{ID})$, then output a decryption key $\mathsf{dk}_{\mathsf{ID}} := (\sigma_1, \sigma_2, \sigma_3)$.
- $\mathsf{Encrypt}_{\mathsf{Wat}}(\mathsf{pk}, \mathsf{ID}, m)$: parse identity as $\mathsf{ID} = (id_1, id_2, \ldots, id_\ell)$, sample $t \leftarrow\!\$\ \mathbb{Z}_q$, compute $C := (e(A, B)^t \cdot m, g_2^t, (u_0 \prod_{i=1}^{\ell} u_i^{id_i})^t)$, then output a ciphertext C.
- $\mathsf{Decrypt}_{\mathsf{Wat}}(\mathsf{dk}_{\mathsf{ID}}, \mathsf{C})$: parse $(\mathsf{dk}_1, \mathsf{dk}_2, \mathsf{dk}_3) := \mathsf{dk}_{\mathsf{ID}}$, ciphertext as $(\mathsf{C}_1, \mathsf{C}_2, \mathsf{C}_3) := \mathsf{C}$, compute $m := \mathsf{C}_1 \cdot e(\mathsf{C}_3, \mathsf{dk}_3) \cdot e(\mathsf{dk}_1, \mathsf{C}_2)^{-1}$, then output a message m.

n-Use Key Exchange (nUKE). n-Use Key Exchange (nUKE) is formally introduced in [45]. nUKE provides a primitive that ensures that given an nUKE attested count-limited key pair, the shared key can only be computed a limited number of times. nUKE requires key to be generated with attestation for non-extractiblity and limited-usage attributes. The security of nUKE is defined as an all-or-nothing distinguishing game on a set of real-or-random keys and rely on the assumptions given below. We extend the assumptions, by incorporating an intermediate assumption 2 that can be placed in between TEE-backed keys and hardware-backed keys. The nUKE and associated assumptions are defined below:

Definition 3.2. *n-Use Key Establishment (nUKE) consists of the following algorithms:*
$\mathsf{Setup}_{\mathsf{Att}}(\lambda)$: *on input security parameter λ, the attestation setup algorithm outputs generation key $\mathsf{gk}_{\mathsf{Att}}$ and verification key $\mathsf{vk}_{\mathsf{Att}}$.*
$\mathsf{KeyGen}(\lambda)$: *on input security parameters λ, the key generation algorithm outputs a key pair $(\mathsf{sk}, \mathsf{pk})$.*
$\mathsf{KeyGen}_{\mathsf{Att}}(\mathsf{gk}_{\mathsf{Att}}, n)$: *on input generation key $\mathsf{gk}_{\mathsf{Att}}$, usage value n, the algorithm outputs a key pair $(\mathsf{sk}, \mathsf{pk})$ and attestation σ_{att}.*
$\mathsf{Shared}(\mathsf{sk}_A, \mathsf{pk}_B)$: *on input secret key sk_A and public key pk_B, the algorithm outputs a shared secret key K from key space $\mathcal{K}$.*
$\mathsf{Verify}_{\mathsf{Att}}(\mathsf{pk}, n, \mathsf{vk}_{\mathsf{Att}}, \sigma_{\mathsf{att}})$: *on input public key pk, usage value n, verification key $\mathsf{vk}_{\mathsf{Att}}$ and attestation σ_{att}, the attestation verification algorithm outputs 1 if pk was generated using $\mathsf{KeyGen}_{\mathsf{Att}}(\mathsf{gk}_{\mathsf{Att}}, n)$, or 0 otherwise.*

Assumption 1 (TEE-Secured Keys). *Private keys inside a TEE-backed hardware keystore and the TEE-backed attestation mechanism are secure against system-level privilege adversaries (e.g., malware) except ones targetting shared hardware using side-channel, fault injection, and other attacks.*

Assumption 2 (Post-Compromise TEE-secured Keys). *Private keys inside a temper-resistance hardware keystore are secure against system-level privilege adversaries (e.g., malware) while attestation mechanism is managed by TEE.*

Assumption 3 (Tempered-resistance Hardware Keys). *Private keys inside a temper-resistance hardware keystore with attestation mechanism are secure against system-level privilege adversaries.*

4 One-Shot Re-Delegatable Signatures

To formally define the notion of one-shot re-delegatable signatures, we modify the syntax proposed by Krenn and Lorünser [25] and Yeoh, Hanzlik, and Valta [45] for one-time delegation primitives. Moreover, we define our primitive with respect to an existing signature scheme with signing capabilities the delegator wants to delegate. More formally.

Definition 4.1. *A one-shot re-delegatable signature scheme* ORBIT *is defined with respect to a signature scheme* DS := (KeyGen, Sign, Verify) *consists of the following set of algorithms:*
$(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{OKeyGen}(\lambda, \mathsf{sk}_{\mathsf{DS}}, \mathsf{vk}_{\mathsf{DS}})$: *on input the security parameter λ, signing key $\mathsf{sk}_{\mathsf{DS}}$, and verification key $\mathsf{vk}_{\mathsf{DS}}$, the one-shot key generation algorithm outputs secret key* sk *and public key* pk.
$(\mathsf{pk}_D, \mathsf{sk}_D) \leftarrow \mathsf{DKeyGen}(\lambda)$: *on input the security parameter λ, the delegatee key generation algorithm outputs delegatee secret key sk_D and public key pk_D.*
$(\mathsf{dmk}, \mathsf{tag}) \leftarrow \mathsf{Delegate}(\mathsf{sk}, \mathsf{pk}, \mathsf{pk}_D)$: *on input delegator secret key* sk*, delegator public key* pk *and delegatee public key pk_D, the delegate algorithm outputs a dormant key* dmk *and an associated* tag.
$\mathsf{ak} \leftarrow \mathsf{Unlock}(\mathsf{sk}, \mathsf{tag})$: *on input signing key* sk *and tag* tag*, the unlock algorithm outputs an active key* ak *associated with* tag.
$\mathsf{osk}/\perp \leftarrow \mathsf{Activate}(\mathsf{dmk}, \mathsf{ak}, \mathsf{pk}, \mathsf{tag})$: *on input a one-shot signing key* dmk*, an activation key* ak*, a public key* pk *and a tag* tag*, the activation algorithm outputs a one-shot secret key* osk *or* $\perp$.
$\mathsf{ak} \leftarrow \mathsf{RDelegate}(\mathsf{osk}, \mathsf{sk}_D, \mathsf{tag})$: *on input one-shot signing key* osk *and tag* tag*, the re-delegate algorithm outputs an activation key* ak *associated with* tag.
$\sigma \leftarrow \mathsf{DSign}(\mathsf{osk}, \mathsf{sk}_D, m)$: *on input one-time signing key* osk*, delegatee secret key sk_D and message m, the delegated sign algorithm outputs a signature σ.*

Some schemes might require public parameters crs*. In such a case, we assume that those parameters are an implicit argument to all algorithms and are generated by a* $\mathsf{Setup}(\lambda)$ *algorithm.*

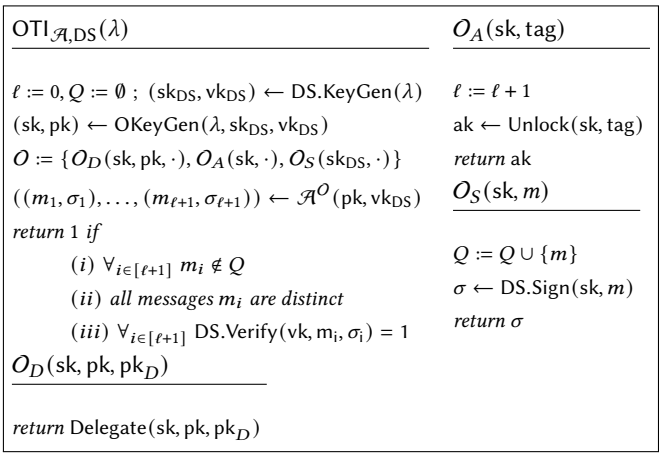
Definition 4.2 (Correctness). *One-shot re-delegatable signature (*ORBIT*) scheme with respect to a signature scheme* DS *is correct if for all security parameters λ, all key pairs* $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{OKeyGen}(\lambda, \mathsf{sk}_{\mathsf{DS}}, \mathsf{vk}_{\mathsf{DS}})$ *where* $(\mathsf{sk}_{\mathsf{DS}}, \mathsf{vk}_{\mathsf{DS}}) \leftarrow \mathsf{DS.KeyGen}(\lambda)$*, all messages m, any number of delegatees n, all delegatee key pairs* $(\mathsf{sk}_{D,i}, \mathsf{pk}_{D,i}) \leftarrow \mathsf{DKeyGen}(\lambda)$ *for $i \in [n]$ we have:*

- $\mathsf{DS.Verify}(\mathsf{vk}_{\mathsf{DS}}, \mathsf{m}, \mathsf{DS.Sign}(\mathsf{sk}_{\mathsf{DS}}, \mathsf{m})) = 1$*, and*
- $\mathsf{DS.Verify}(\mathsf{vk}_{\mathsf{DS}}, \mathsf{m}, \mathsf{DSign}(\mathsf{osk}_{\mathsf{n}}, \mathsf{sk}_{\mathsf{D,n}}, \mathsf{m})) = 1$

where $\mathsf{ak}_{\mathsf{tag},1} \leftarrow \mathsf{Unlock}(\mathsf{sk}, \mathsf{tag}_1)$ *and* $\forall_{i \in [n]}$:

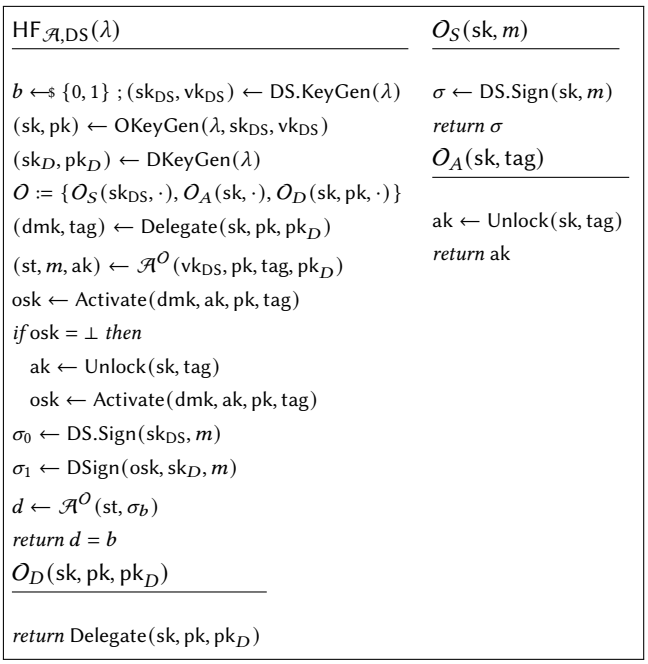
- $(\mathsf{dmk}_{\mathsf{tag},i}, \mathsf{tag}_i) \leftarrow \mathsf{Delegate}(\mathsf{sk}, \mathsf{pk}, \mathsf{pk}_{D,i})$
- $\mathsf{osk}_i \leftarrow \mathsf{Activate}(\mathsf{dmk}_{\mathsf{tag},i}, \mathsf{ak}_{\mathsf{tag},i}, \mathsf{pk}, \mathsf{tag}_i)$
- $(\mathsf{ak}_{\mathsf{tag},i+1}) \leftarrow \mathsf{RDelegate}(\mathsf{osk}_i, \mathsf{sk}_{D,i}, \mathsf{tag}_{i+1})$

Definition 4.3 (One-timeness). *One-shot re-delegatable signature (*ORBIT*) scheme with respect to a signature scheme* DS *is said to satisfy one-timeness if for all $\lambda \in \mathbb{N}$, it holds that every PPT adversary $\mathcal{A}$ has at most negligible advantage in the following experiment, where advantage is defined as* $\mathbf{Adv}_{\mathsf{OTI}}(\mathcal{A}) := \Pr[\mathsf{OTI}_{\mathcal{A},\mathsf{DS}}(\lambda) = 1]$.



One-timeness holds only with respect to the delegates, and the delegator (i.e., owner of the signing key for DS) can still sign without restrictions. Similarly, below, we define the notion of history-freeness, which informally states that signatures created via delegation look like signatures created by DS.Sign. Consequently, delegated signatures do not contain information about the parties involved in the delegation process. However, this only holds with respect to third parties since the delegator is not considered an adversary here and could potentially identify the delegatee that creates the signature using DSign. We leave a stronger notion that protects even against a malicious delegator for future work.

Definition 4.4 (History-freeness). *One-shot re-delegatable signature (*ORBIT*) scheme with respect to a signature scheme* DS *is said to be history-free if for all $\lambda \in \mathbb{N}$, it holds that every PPT adversary $\mathcal{A}$ has at most negligible advantage in the following experiment, where advantage is defined as* $\mathbf{Adv}_{\mathsf{HF}}(\mathcal{A}) := |\Pr[\mathsf{HF}_{\mathcal{A},\mathsf{DS}}(\lambda) = 1] - 1/2|$.



4.1 Verifiability

In many applications, the delegator might be malicious and provide clients with invalid dormant keys dmk. Thus, we additionally want the notion of verifiability for ORBIT, and for this, we introduce the following algorithm to check if for a given dormant key, it can be activated to a valid one-shot signing key osk:
$0/1 \leftarrow \mathsf{VerifyD}(\mathsf{dmk}, \mathsf{tag}, \mathsf{pk}, \mathsf{pk}_D)$: *on input the dormant key* dmk, *the tag* tag, *a public key* pk, *and a delegatee public key* pk_D, *the delegation verification algorithm outputs* 1 *if the dormant key is valid, and* 0 *otherwise.*

Definition 4.5 (Verifiability). *One-shot re-delegatable signature (*ORBIT*) scheme with respect to a signature scheme* DS *is said to be verifiable if for all* $\lambda \in \mathbb{N}$, *it holds that every PPT adversary* $\mathcal{A}$ *has at most negligible advantage in the following experiment, where we define the advantage as* $\mathbf{Adv}_{\mathsf{VER}}(\mathcal{A}) := \Pr[\mathsf{VER}_{\mathcal{A},\mathsf{DS}}(\lambda) = 1]$.

$\mathsf{VER}_{\mathcal{A},\mathsf{DS}}(\lambda)$

$(\mathsf{pk}_D, \mathsf{sk}_D) \leftarrow \mathsf{DKeyGen}(\lambda)$; $(\mathsf{pk}_S, \mathsf{sk}_S) \leftarrow \mathsf{DKeyGen}(\lambda)$
$((\mathsf{dmk}_S, \mathsf{tag}_S), (\mathsf{dmk}_D, \mathsf{tag}_D), \mathsf{ak}_D, \mathsf{pk}) \leftarrow \mathcal{A}(\lambda, \mathsf{pk}_S, \mathsf{pk}_D)$
if $\mathsf{VerifyD}(\mathsf{dmk}_D, \mathsf{tag}_D, \mathsf{pk}, \mathsf{pk}_D) = 0$ *return* 0
if $\mathsf{VerifyD}(\mathsf{dmk}_S, \mathsf{tag}_S, \mathsf{pk}, \mathsf{pk}_S) = 0$ *return* 0
$\mathsf{osk}_D \leftarrow \mathsf{Activate}(\mathsf{dmk}_D, \mathsf{ak}_D, \mathsf{pk}, \mathsf{tag}_D)$
if $\mathsf{osk}_D = \perp$ *return* 0
$\mathsf{ak}_S \leftarrow \mathsf{RDelegate}(\mathsf{osk}_D, \mathsf{sk}_D, \mathsf{tag}_D)$
$\mathsf{osk}_S \leftarrow \mathsf{Activate}(\mathsf{dmk}_S, \mathsf{ak}_S, \mathsf{pk}, \mathsf{tag}_S)$
$m \leftarrow\!\$\ \mathcal{M}$
if $\mathsf{osk}_S = \perp \vee \mathsf{DS.Verify}(\mathsf{vk}, \mathsf{DSign}(\mathsf{osk}_S, \mathsf{sk}_S, \mathsf{m}), \mathsf{m}) = 0$
  return 1
else return 0

5 Construction

In this section, we define our ORBIT scheme. The core idea on how to construct ORBIT efficiently is discussed in detail in Section 2.3. In short, instead of constructing Yao's garble circuit, we bit decompose the signature into randomized message-dependent shares p_i and non-message dependent part p_0 such that to derandomize and obtain the correct signature, delegatee have to assemble all parts together. We make use of the shared key from the n-use key establishment as a one-time pad in an all-or-nothing approach to share only the correct values with the delegatee as in [45]. The delegatee can only decrypt exactly $n = \ell_m + 1$ values, one for each bit of the message/tag, plus a path choice. We secret share a value a and encode the same share in the ciphertext of a given position for both bits. An adversary that decrypts the same bit-position twice will not be able to reconstruct the shared value a, thus losing the ability to decrypt ciphertexts that encode the corresponding chosen signature. In addition, we also encrypt both garbled circuits with an extra shared key from nUKE so that the delegatee can only choose exactly one path of the garbled circuit to evaluate. Finally, we hide the final secret share of a, a_{tag} inside a IBE encrypted ciphertext under the identity tag. This ensures that the circuit cannot be evaluated without having the corresponding activation key $\mathsf{ak}_{\mathsf{tag}}$.

$\underline{\mathsf{OKeyGen}(\lambda, \mathsf{sk}_{\mathsf{ECDSA}}, \mathsf{pk}_{\mathsf{ECDSA}})}$:
- Sample $(\mathsf{sk}_{\mathsf{Wat}}, \mathsf{pk}_{\mathsf{Wat}}) \leftarrow \mathsf{Setup}_{\mathsf{Wat}}(\lambda)$.
- Set $\mathsf{sk} := (\mathsf{sk}_{\mathsf{ECDSA}}, \mathsf{sk}_{\mathsf{Wat}})$ and $\mathsf{pk} := (\mathsf{pk}_{\mathsf{ECDSA}}, \mathsf{pk}_{\mathsf{Wat}})$.
- Return $(\mathsf{sk}, \mathsf{pk})$.

$\underline{\mathsf{DKeyGen}(\lambda)}$:
- Execute $(\mathsf{sk}_{\mathsf{nUKE}}, \mathsf{pk}_{\mathsf{nUKE}}, \sigma_{\mathsf{att}}) \leftarrow \mathsf{nUKE.KeyGen}_{\mathsf{Att}}(\mathsf{gk}_{\mathsf{Att}}, \ell_m + 1)$.
  // Here ℓ_m denotes the length of the message
- set $\mathsf{sk}_D := \mathsf{sk}_{\mathsf{nUKE}}$ and $\mathsf{pk}_D := (\mathsf{pk}_{\mathsf{nUKE}}, \sigma_{\mathsf{att}})$.
- Return $(\mathsf{sk}_D, \mathsf{pk}_D)$.

$\underline{\mathsf{Unlock}(\mathsf{sk}, \mathsf{tag})}$:
- Parse $(\mathsf{sk}_{\mathsf{ECDSA}}, \mathsf{sk}_{\mathsf{Wat}}) := \mathsf{sk}$.
- Compute $\mathsf{dk}_{\mathsf{tag}} := \mathsf{KeyDer}_{\mathsf{Wat}}(\mathsf{sk}_{\mathsf{Wat}}, \mathsf{tag})$.
- Return $\mathsf{ak}_{\mathsf{tag}} := \mathsf{dk}_{\mathsf{tag}}$.

$\underline{\mathsf{Activate}(\mathsf{dmk}, \mathsf{ak}, \mathsf{pk}, \mathsf{tag})}$:
- Parse $\mathsf{dmk} := (\mathcal{G}_{\mathsf{ECDSA}}, \mathcal{G}_{\mathsf{Wat}}, \mathcal{G}_{\mathcal{S}})$, $(c_{\mathsf{ak}}, \omega) := \mathcal{G}_{\mathcal{S}}$, and $(\cdot, \mathsf{pk}_{\mathsf{Wat}}) := \mathsf{pk}$.
- Compute $r_{\mathsf{tag}} := \mathsf{Decrypt}_{\mathsf{Wat}}(\mathsf{ak}_{\mathsf{tag}}, c_{\mathsf{ak}})$.
- Set $\widehat{\mathcal{G}_{\mathcal{S}}} := (r_{\mathsf{tag}}, \omega)$.
- Return $\perp$ if $m \neq \mathsf{Decrypt}_{\mathsf{Wat}}(\mathsf{ak}_{\mathsf{tag}}, \mathsf{Encrypt}_{\mathsf{Wat}}(pk_{\mathsf{Wat}}, \mathsf{tag}, m))$ where $m \leftarrow\!\$\ \mathbb{G}_T$, else return $\mathsf{osk} := (\mathcal{G}_{\mathsf{ECDSA}}, \mathcal{G}_{\mathsf{Wat}}, \widehat{\mathcal{G}_{\mathcal{S}}})$.

$\underline{\mathsf{DSign}(\mathsf{osk}, \mathsf{sk}_D, m)}$:
- Parse $(\mathcal{G}_{\mathsf{ECDSA}}, \cdot, \widehat{\mathcal{G}_{\mathcal{S}}}) := \mathsf{osk}$, $\Big(\widehat{c_r}, p_0, (\widehat{d_{i,0}}, \widehat{d_{i,1}})_{i\in[\ell_m]}\Big) := \mathcal{G}_{\mathsf{ECDSA}}$ and $\Big(r_{\mathsf{tag}}, (\mathsf{pk}_{\mathcal{S},0}, c_{\mathcal{S},0}, \cdot), (\mathsf{pk}_{i,0}, c_{i,0}, \mathsf{pk}_{i,1}, c_{i,1})_{i\in[\ell_m]}\Big) := \widehat{\mathcal{G}_{\mathcal{S}}}$.
- Use $h_1 \ldots h_{\ell_m}$ to denote the ℓ_m bits of $\mathsf{H}(m)$.
- Compute: // Recomputing the branch specific secret $a_{\mathcal{S},0}$
$$K_{\mathcal{S},0} \leftarrow \mathsf{nUKE.Shared}(\mathsf{sk}_D, \mathsf{pk}_{\mathcal{S},0})$$
$$a_{\mathcal{S},0} := \mathsf{H}_2(K_{\mathcal{S},0}) \oplus c_{\mathcal{S},0}.$$
- For each $i \in [\ell_m]$ compute: // Computing keys K_{i,h_i} corresponding to the inputs h_i and decrypting shares a_i and b_{i,h_i}
$$K_{i,h_i} \leftarrow \mathsf{nUKE.Shared}(\mathsf{sk}_D, \mathsf{pk}_{i,h_i})$$
$$(a_i || b_{i,h_i}) := \mathsf{H}_2(K_{i,h_i}) \oplus c_{i,h_i}.$$
$$d_{i,h_i} := \mathsf{H}_2(a_{\mathcal{S},0} || i, h_i) \oplus \widehat{d_{i,h_i}}.$$
- Compute $c_r := \mathsf{H}_2(a_{\mathcal{S},0}) \oplus \widehat{c_r}$ and $a_{\mathsf{tag}} := \mathsf{H}_2(r_{\mathsf{tag}})$.
- Compute secret $a_{sum} := \oplus_{i\in[\ell_m]} a_i$, $a := a_{sum} \oplus a_{\mathsf{tag}}$, and for each $i \in [\ell_m]$, compute $p_i := \mathsf{H}_2(a || b_{i,h_i}) \oplus d_{i,h_i}$.
- Compute $s := p_0 + \sum_{i\in[\ell_m]} p_i$, $r := c_r \oplus \mathsf{H}_2(a)$
- Return $\sigma := (r, s)$.

Figure 3: Key generation, unlocking, activation and delegated signature generation algorithms of ORBIT**.**

In the discussed solution, the delegator is using a bit decomposition of the message to compute the shares t_i and $t_i + p_i$ which is later used to compute the final signature on the message $\mathsf{H}(m)$. For efficiency purposes, we could consider using word-size decomposition $w > 1$ rather than bit decomposition $w = 1$. This directly leads to ℓ_m/w shares meaning the number of computations done by the

client's hardware is shrunk by a factor of w, albeit with an increase of computational for the delegator. The improvement is shown concretely in Table 1. The same approach can be applied analogously to Waters decryption key for RDelegate. For the remaining of the construction, we focus on $w = 1$ as $w = 8$ apply analogously.

Let the group used in ECDSA be a group of prime order-q, the group used in Waters scheme be a group of prime order q', ℓ_m be the bit length of tag and message m. Let $\mathsf{H}_2 : \{0,1\}^* \rightarrow \{0,1\}^n$ be a random oracle. $\mathsf{crs} := (\mathsf{vk}_{\mathsf{Att}}, g_1, g_2, g, q, q', \ell_m)$ where $(\mathsf{vk}_{\mathsf{Att}}, \mathsf{gk}_{\mathsf{Att}}) \leftarrow \mathsf{Setup}_{\mathsf{Att}}(\lambda)$ is executed as part of the trusted hardware setup. $\mathsf{gk}_{\mathsf{Att}}$ is distributed to the delegatee as part of the trusted hardware key provisioning process. crs is implicitly given for the algorithms defined. We construct a one-shot re-delegatable signature scheme ORBIT := (OKeyGen, DKeyGen, Delegate, Unlock, Activate, RDelegate, DSign) as described in Figure 3 and 4. We rely on ECDSA signature scheme, Waters signature scheme, Waters IBE, and n-Use key establishment scheme (see Section 3) as the building blocks.

5.1 Verifiability

To achieve verifiability in ORBIT, we extend the scheme defined in Section 5 to $\mathsf{ORBIT}_{\mathsf{ZK}}$ with a modified $\mathsf{Delegate}_{\mathsf{ZK}}$ and an additional algorithm VerifyD. In $\mathsf{Delegate}_{\mathsf{ZK}}$, we additionally prove the well-formness of the provided dmk with the relation $\mathcal{R}_{\mathsf{dmk}}(x, w) =$

$$\left\{ \begin{array}{c} (\mathsf{dmk}, \mathsf{tag}, \mathsf{pk}, \mathsf{pk}_D), \\ (\mathsf{sk}, W) \end{array} : \begin{array}{c} (\mathsf{pk}_1, \mathsf{pk}_2) := \mathsf{pk} \wedge \\ (\mathsf{sk}_1, \mathsf{sk}_2) := \mathsf{sk} \wedge \\ (\mathsf{sk}_1, \mathsf{pk}_1) \leftarrow \mathsf{KeyGen}_{\mathsf{ECDSA}}(\lambda) \wedge \\ (\mathsf{sk}_2, \mathsf{pk}_2) \leftarrow \mathsf{Setup}_{\mathsf{Wat}}(\lambda)\} \wedge \\ (\mathsf{dmk}, \mathsf{tag}) \leftarrow \mathsf{Delegate}(\mathsf{sk}, \mathsf{pk}, \\ , \mathsf{pk}_D) \text{ where } W \\ \text{are used in Delegate} \end{array} \right\}.$$

For the sake of conciseness, W is used to represent all randomly chosen elements and ephemeral keys used in the Delegate algorithm. On receiving a dmk from the issuer, the participant runs VerifyD with the included proof π to either accept or reject the provided dormant key. Using a zero-knowledge argument system, the delegator can prove to the delegatee that the output of Delegate satisfies the relation $\mathcal{R}_{\mathsf{dmk}}(x, w)$, allowing the delegatee to confirm that the dormant key dmk is correctly computed such that it can be later transformed into a correct activation key ak or a correct signature σ. Modifications to the algorithms are given below:

$\underline{\mathsf{Delegate}_{\mathsf{ZK}}(\mathsf{sk}, \mathsf{pk}, \mathsf{pk}_D) : \mathsf{Run}\ (\mathsf{dmk}, \mathsf{tag}) \leftarrow \mathsf{Delegate}(\mathsf{sk}, \mathsf{pk}, \mathsf{pk}_D)}$ and $\pi \leftarrow \mathsf{NIZK.Prove}(\mathsf{crs}, (\mathsf{dmk}, \mathsf{tag}, \mathsf{pk}, \mathsf{pk}_D), (\mathsf{sk}, W))$ where W represents all sampled random elements and ephemeral keys used by Delegate. Set $\mathsf{dmk}' := (\mathsf{dmk}, \pi)$. Return $(\mathsf{dmk}', \mathsf{tag})$.

$\underline{\mathsf{VerifyD}_{\mathsf{ZK}}(\mathsf{dmk}, \mathsf{tag}, \mathsf{pk}, \mathsf{pk}_D) : \mathsf{Parse}\ (\mathsf{dmk}, \pi) := \mathsf{dmk}. \mathsf{Return}\ \mathsf{NIZK}.}$
$\mathsf{Verify}(\mathsf{crs}, (\mathsf{dmk}, \mathsf{tag}, \mathsf{pk}, \mathsf{pk}_{\mathsf{D}}), \pi)$.

One generic way of proving an arbitrary NP-relation is to convert the relation into an arithmetic circuit and use existing generic zkSNARK [8] prove system such as Groth16 [20] and Spartan [37] to produce and verify the proof. zkSNARK has the benefit of succinct proof size, but it would incur a non-trivial amount of computation overhead to the prover, especially when the computation of cryptographic operation is in a non-native proving field. In this specific case, the computations to be proved are split into two different fields. One of them is a hardware-supported standardized named curve for key exchange and another one is a pairing-friendly curve for Waters scheme. We estimated the number of non-linear constraints based on the dominating operations (non-native field operations) to be around 5 million, which would cost around $15-20s$ of prover time. It is worth noting that this zero-knowledge proof is computed during the offline phase and does not influence the transaction time in the payment application of ORBIT. To achieve better prover time, we could split the SNARK circuit into different sub-circuits, whereby the shared variables are committed using a field-independent commitment scheme such as hash commitment, and provide the opening for it in other sub-circuits. This would allow the SNARK to prove the statement in the native field, which is significantly faster than non-native field operations. By using zkSNARK such as Spartan that supports any elliptic curve that the discrete log problem is hard in, then it is feasible to achieve the aforementioned optimization. Furthermore, the prove generation can be partially batch processed since only the relation on the encryption of $a||b_i$ material depends on the delegatee's input.

Alternatively, we could employ the cut-and-choose technique instead of zkSNARK to enforce the correctness of Delegate computation, albeit with a larger communication size as a tradeoff. The signature scheme is changed from ECDSA to Waters signature effectively condensing DSign and RDelegate circuits into one. The main idea is to let the delegator provide $2 \cdot \lambda$ instances of dmk-circuit generated with a seed sd using $\mathsf{Delegate}_{\mathsf{sd}}$. The signer secret key sk is then secret shared using $\lambda + 1$ out of $2 \cdot \lambda$ verifiable secret sharing (VSS) [16] and the shares are now the secret signing key for the dmk instances. The verifier then randomly samples a subset of instances of size λ to check for correctness. The delegator now reveals λ of the secret key shares of sk. The verifier can check the correctness of the shares and the circuit accordingly. In the online phase, the verifier evaluates the remaining circuits to recover a correct signature under a valid share which can be reconstructed to the signature under the original secret-shared signing key. Extra care must be taken to ensure that the message chosen is simultaneously binding to all the circuits. We leave the concrete instantiation of an efficient ORBIT with verifiability as future work.

5.2 Security Analysis

Lemma 5.1 (Correctness). ORBIT *is a correct one-shot re-delegatable signature scheme, assuming* nUKE, ECDSA *and* Wat *are correct.*

Theorem 5.2. ORBIT *is one-time secure in the random oracle model, assuming* nUKE *scheme is n-use real-or-random key secure, Waters IBE scheme is IND-ID-CPA secure and the ECDSA signature scheme is unforgeable under chosen message attacks with "raw" signing queries [21].*

Theorem 5.3. ORBIT *and* $\mathsf{ORBIT}_{\mathsf{ZK}}$ *are perfect history-free one-shot re-delegatable siganture schemes.*

Theorem 5.4. $\mathsf{ORBIT}_{\mathsf{ZK}}$ *is one-time secure in the random oracle model, assuming* nUKE *scheme is n-use real-or-random key secure, Waters IBE scheme is IND-ID-CPA secure, the proof system is zero-knowledge and the ECDSA signature scheme is unforgeable under chosen message attacks with "raw" signing queries [21].*

Theorem 5.5. $\mathsf{ORBIT}_{\mathsf{ZK}}$ *is a verifiable one-shot re-delegatable signature scheme, assuming the proof system used is sound.*

Delegate$(\mathsf{sk}, \mathsf{pk}, \mathsf{pk}_D)$:

- Parse $(\mathsf{sk}_{\mathsf{ECDSA}}, \mathsf{sk}_{\mathsf{Wat}}) := \mathsf{sk}$, $(\mathsf{pk}_{\mathsf{ECDSA}}, \mathsf{pk}_{\mathsf{Wat}}) := \mathsf{pk}$ and $(\mathsf{pk}_N, \sigma_{\mathsf{att}}) := \mathsf{pk}_D$.
- If $\mathsf{Verify}_{\mathsf{Att}}(\mathsf{pk}_{\mathsf{nUKE}}, \ell_m + 1, \mathsf{vk}_{\mathsf{Att}}, \sigma_{\mathsf{att}}) = 0$ then return $\perp$. // verifying that pk_D is a limited-count key
- Sample $\mathsf{tag} \leftarrow\!\$\ \{0,1\}^{\ell_m}$, $a_{\mathcal{S},0}, a_{\mathcal{S},1} \leftarrow\!\$\ \{0,1\}^{\ell_m/2}$.
- Sample $a_{\mathsf{tag}} := \mathsf{H}_2(r_{\mathsf{tag}})$ where $r_{\mathsf{tag}} \leftarrow\!\$\ \mathbb{G}_T$. // Computing activation share a_{tag} as a hash of plaintext r_{tag} of the Waters IBE
- Choose $a_1, b_{1,0}, b_{1,1}, \ldots, a_{\ell_m}, b_{\ell_m,0}, b_{\ell_m,1} \leftarrow\!\$\ \{0,1\}^{\ell_m/2}$. Compute $a_{sum} := \bigoplus_{i\in[\ell_m]} a_i$, $a \leftarrow a_{sum} \oplus a_{\mathsf{tag}}$.
- Compute the ciphertext $c_{\mathsf{ak}} \leftarrow \mathsf{Encrypt}_{\mathsf{Wat}}(\mathsf{pk}_{\mathsf{Wat}}, \mathsf{tag}, r_{\mathsf{tag}})$.
- For all $i \in \{0,1\}$: // Encrypting branch specific secrets, $a_{\mathcal{S},0}$ for DSign and $a_{\mathcal{S},1}$ for RDelegate

$$(\mathsf{sk}_{\mathcal{S},i}, \mathsf{pk}_{\mathcal{S},i}) \leftarrow \mathsf{nUKE.KeyGen}(\lambda), \quad K_{\mathcal{S},i} \leftarrow \mathsf{nUKE.Shared}(\mathsf{sk}_{\mathcal{S},i}, \mathsf{pk}_N), \quad c_{\mathcal{S},i} := \mathsf{H}_2(K_{\mathcal{S},i}) \oplus (a_{\mathcal{S},i}).$$

- For all $i \in [\ell_m]$: // Encrypting secrets $b_{i,0}$ and $b_{i,1}$ corresponding to the users choice for the i-th input and shares a_i of the all-or-nothing secret a

$$(\mathsf{sk}_{i,0}, \mathsf{pk}_{i,0}) \leftarrow \mathsf{nUKE.KeyGen}(\lambda), \quad K_{i,0} \leftarrow \mathsf{nUKE.Shared}(\mathsf{sk}_{i,0}, \mathsf{pk}_N), \quad c_{i,0} := \mathsf{H}_2(K_{i,0}) \oplus (a_i || b_{i,0})$$

$$(\mathsf{sk}_{i,1}, \mathsf{pk}_{i,1}) \leftarrow \mathsf{nUKE.KeyGen}(\lambda), \quad K_{i,1} \leftarrow \mathsf{nUKE.Shared}(\mathsf{sk}_{i,1}, \mathsf{pk}_N), \quad c_{i,1} := \mathsf{H}_2(K_{i,1}) \oplus (a_i || b_{i,1})$$

- Compute $t_0 := \sum_{i\in[\ell_m]}(-1)\cdot t_i$, where $t_1, \ldots, t_{\ell_m} \leftarrow\!\$\ \mathbb{Z}_q^*$. // Sharing of 0 for ECDSA garbling
- Choose $k \leftarrow\!\$\ \mathbb{Z}_q^*$, compute $(x, y) = g^k$ and set $r := x \mod q$. // Randomness sampling for ECDSA
- Set $p_0 := k^{-1} \cdot r \cdot \mathsf{sk}_{\mathsf{ECDSA}} + t_0$, $c_r := \mathsf{H}_2(a) \oplus r$, and $\widehat{c_r} := \mathsf{H}_2(a_{\mathcal{S},0}) \oplus c_r$.
- For all $i \in [\ell_m]$: // Garbling of the ECDSA signature with payloads $p_{i,0}$ and $p_{i,1}$

$$p_{i,0} := t_i, \qquad d_{i,0} := \mathsf{H}_2(a||b_{i,0}) \oplus p_{i,0}, \quad \widehat{d_{i,0}} := \mathsf{H}_2(a_{\mathcal{S},0}||i, 0) \oplus d_{i,0}$$

$$p_{i,1} := 2^{i-1} \cdot k^{-1} + t_i, \quad d_{i,1} := \mathsf{H}_2(a||b_{i,1}) \oplus p_{i,1}, \quad \widehat{d_{i,1}} := \mathsf{H}_2(a_{\mathcal{S},0}||i, 1) \oplus d_{i,1}$$

- Compute $t'_0 := \prod_{i\in[\ell_m]}(t'_i)^{-1}$, where $t'_1, \ldots, t'_{\ell_m} \leftarrow\!\$\ \mathbb{G}_1$. // Sharing of $1_{\mathbb{G}_1}$ for Waters garbling
- Sample $\gamma \leftarrow\!\$\ \mathbb{Z}_{q'}^*$ and compute: // Computing payload part p'_0 and encryption of $\sigma_2 = g_1^r$ and $\sigma_3 = g_2^r$ of Waters signature

$$p'_0 \leftarrow (\mathsf{sk}_{\mathsf{Wat}} \cdot u_0^{\gamma} \cdot t'_0), \quad c_\gamma \leftarrow \left(\mathsf{H}_2(a||1) \oplus g_1^{\gamma}, \mathsf{H}_2(a||2) \oplus g_2^{\gamma}\right), \quad \widehat{c_\gamma} \leftarrow \mathsf{H}_2(a_{\mathcal{S},1}) \oplus c_\gamma.$$

- For all $i \in [\ell_m]$: // Garbling of the Waters signature with payloads $p'_{i,0}$ and $p'_{i,1}$

$$p'_{i,0} := t'_i, \qquad d'_{i,0} := \mathsf{H}_2(a||b_{i,0}) \oplus p'_{i,0}, \qquad \widehat{d'_{i,0}} := \mathsf{H}_2(a_{\mathcal{S},1}||i, 0) \oplus d'_{i,0}$$

$$p'_{i,1} := u_i^{\gamma} \cdot t'_i, \qquad d'_{i,1} := \mathsf{H}_2(a||b_{i,1}) \oplus p'_{i,1}, \qquad \widehat{d'_{i,1}} := \mathsf{H}_2(a_{\mathcal{S},1}||i, 1) \oplus d'_{i,1}$$

- Set $\mathcal{G}_{\mathsf{ECDSA}} := \left(\widehat{c_r}, p_0, (\widehat{d_{i,0}}, \widehat{d_{i,1}})_{i\in[\ell_m]}\right)$, $\mathcal{G}_{\mathsf{Wat}} := \left(\widehat{c_\gamma}, p'_0, (\widehat{d'_{i,0}}, \widehat{d'_{i,1}})_{i\in[\ell_m]}\right)$,
  $\mathcal{G}_{\mathcal{S}} := \left(c_{\mathsf{ak}}, (\mathsf{pk}_{\mathcal{S},i}, c_{\mathcal{S},i})_{i\in\{0,1\}}, (\mathsf{pk}_{i,0}, c_{i,0}, \mathsf{pk}_{i,1}, c_{i,1})_{i\in[\ell_m]}\right)$.
- Return $(\mathsf{dmk} := (\mathcal{G}_{\mathsf{ECDSA}}, \mathcal{G}_{\mathsf{Wat}}, \mathcal{G}_{\mathcal{S}}), \mathsf{tag})$.

RDelegate$(\mathsf{osk}, \mathsf{sk}_D, \mathsf{tag})$:

- Parse $(\cdot, \mathcal{G}_{\mathsf{Wat}}, \widehat{\mathcal{G}_{\mathcal{S}}}) := \mathsf{osk}$, $\left(\widehat{c_\gamma}, p'_0, (\widehat{d'_{i,0}}, \widehat{d'_{i,1}})_{i\in[\ell_m]}\right) := \mathcal{G}_{\mathsf{Wat}}$ and $\left(r_{\mathsf{tag}}, (\cdot, \mathsf{pk}_{\mathcal{S},1}, c_{\mathcal{S},1}), (\mathsf{pk}_{i,0}, c_{i,0}, \mathsf{pk}_{i,1}, c_{i,1})_{i\in[\ell_m]}\right) := \widehat{\mathcal{G}_{\mathcal{S}}}$.
- Use $\mathsf{btag}_1, \ldots, \mathsf{btag}_{\ell_m}$ to denote the bit decomposition of tag with bit length ℓ_m.
- Compute: // Recomputing the branch specific secret $a_{\mathcal{S},1}$

$$K_{\mathcal{S},1} \leftarrow \mathsf{nUKE.Shared}(\mathsf{sk}_D, \mathsf{pk}_{\mathcal{S},1}), \quad a_{\mathcal{S},1} := \mathsf{H}_2(K_{\mathcal{S},1}) \oplus c_{\mathcal{S},1}.$$

- For each $i \in [\ell_m]$ compute: // Computing secret keys K_{i,btag_i} corresponding to the inputs bits btag_i and decrypting shares a_i and b_{i,btag_i}

$$K_{i,\mathsf{btag}_i} \leftarrow \mathsf{nUKE.Shared}(\mathsf{sk}_D, \mathsf{pk}_{i,\mathsf{btag}_i}), \quad (a_i||b_{i,\mathsf{btag}_i}) := \mathsf{H}_2(K_{i,\mathsf{btag}_i}) \oplus c_{i,\mathsf{btag}_i}, \quad d'_{i,\mathsf{btag}_i} := \mathsf{H}_2(a_{\mathcal{S},1}||i, \mathsf{btag}_i) \oplus \widehat{d'_{i,\mathsf{btag}_i}}.$$

- Compute $(c_{\gamma,1}, c_{\gamma,2}) := \mathsf{H}_2(a_{\mathcal{S},1}) \oplus \widehat{c_\gamma}$ and $a_{\mathsf{tag}} := \mathsf{H}_2(r_{\mathsf{tag}})$.
- Compute secret $a_{sum} := \oplus_{i\in[\ell_m]} a_i$, $a := a_{sum} \oplus a_{\mathsf{tag}}$, and for each $i \in [\ell_m]$, compute $p'_i := \mathsf{H}_2(a||b_{i,\mathsf{btag}_i}) \oplus d'_{i,\mathsf{btag}_i}$.
- Compute $\mathsf{dk}_2 := \mathsf{H}_2(a||1) \oplus c_{\gamma,1}$,$\mathsf{dk}_3 := \mathsf{H}_2(a||2) \oplus c_{\gamma,2}$ and $\mathsf{dk}_1 := p'_0 \cdot \prod_{i\in[\ell_m]} p'_i$.
- Return $\mathsf{ak} := (\mathsf{dk}_1, \mathsf{dk}_2, \mathsf{dk}_3)$.

Figure 4: Delegation and re-delegation algorithms of ORBIT**.**

The intuition behind the one-time security of ORBIT is that for each delegated ECDSA signature computed by the delegatee, there must exist a genesis activation key issued using Unlock. Therefore, the delegatee cannot compute more signatures than the issuer issues genesis activation keys. Moreover, perfect history-freeness follows from the fact that without access to dmk and osk used by

the challenger, the distribution of signatures computed by DSign and DS.Sign are the same. Both also hold for the $\mathsf{ORBIT}_{\mathsf{ZK}}$ version, where additionally, for one-timeness, we simulate that zero-knowledge proof and use the same arguments as for the standard version. Finally, the verifiability of $\mathsf{ORBIT}_{\mathsf{ZK}}$ follows from the soundness of the proof system. For completeness, we provide the full proofs in Appendix B.

6 Implementation and Evaluation

In this section, we elaborate on the implementation of the proposed scheme ORBIT. The implementation comprises of a delegator server with delegation capability and a delegatee mobile application handling delegated signature. We demonstrate the feasibility of the proposed ORBIT scheme by providing comprehensive evaluations. We evaluate the communication and computational performance and conclude that the proposed scheme is practically efficient. The source code is made open source in an open-source repository [3]. We instantiated nUKE using Android Keystore. Note that strongbox-backed keys could provide assumption 2 or assumption 3 depending on the exact attestation mechanism used. It is worth noting that assumption 2 relies on the smartphone's TEE to take care of the count limit, which, as we discussed, is a stronger assumption than when this is taken care of by a separate chip (as in assumption 3). The Google Pixel 6 we used for evaluation provides only 2. However, we do not expect significant changes in DSign and RDelegate in comparison to smartphones supporting assumption 3 due to the attestation process being an independent process done at key generation. Moreover, to provide a security and efficiency trade-off, we compare the hardware-backed approach with a fully software-based solution where the keys and attestation are done by the smartphone's TEE (assumption 1). Section 2.5 gives other possible devices instantiating those assumptions. The elliptic curve used in key exchange and ECDSA is a well-supported secp256r1 curve, while the pairing-friendly elliptic curve used in the Waters scheme is the commonly used BN254 curve.

The system can be optimized by providing two types of dmk for RDelegate or DSign. With that, ℓ_m can now be separated into two different parameters: bit length of an ECDSA message, $\ell_{\mathsf{ECDSA}} := 256$ and bit length of a tag, $\ell_{\mathsf{tag}} := 40$. By setting $\ell_{\mathsf{tag}} := 40$, the circuit is overall more efficient and compact without affecting the security. Note that this parameter corresponds to re-delegation and the tag for the dormant key. In other words, we only use 40-bit of choice for users to pick the message in the Waters signature, considering that this is a reasonable space of tags for the system's life-cycle without any key rotation, i.e., the space of tags is 2^{40}.

6.1 Performance Analysis

The implementation is benchmarked on a 2021 Google Pixel 6 Pro device running on an Android operating system and a 2023 M3 Pro Macbook. The provided implementation is only a proof-of-concept, leaving room for further optimizations in terms of efficiency improvements. Any one-use keys supported device can be extended to support ORBIT by sending over n-numbered of one-use keys (and their attestations), which increases the computation time of DKeyGen by a factor of n while RDelegate and DSign remain relatively unchanged. Therefore, the benchmark result in Table 1 only considers the n-use key case.

[3]Available at https://anonymous.4open.science/r/orbit-7C66/

Table 1: Performance overview for ORBIT implementation on Google Pixel 6 Pro. Time is averaged over 100 executions. TEE denotes a software-backed Android Keystore where the Android OS ensures the security of the nUKE primitive. StrongBox denotes the hardware-backed Android Keystore with full hardware-enforced nUKE. w denotes the size of the input labels 2^w (e.g., for $w = 1$ we use bit as inputs).

Operation	Config	Time (ms)	
		$w = 1$	$w = 8$
Delegate	$\ell_{\mathsf{tag}} = 40$	15.90	300.98
	$\ell_{ecdsa} = 256$	46.96	692.17
DKeyGen	TEE	14.46	
	StrongBox	140.78	
RDelegate	TEE	386.36	116.35
	StrongBox	2038.68	333.9
DSign	TEE	2215.36	337.31
	StrongBox	12606.65	1671.79

Performing DKeyGen on the client side using TEE (software-backed keys with Android OS ensuring the security of nUKE) and StrongBox (hardware-backed keys) has minimal overhead, which takes approximately 14 ms and 140 ms, respectively. Delegation by the delegator server only incurs the negligible computational overhead of around $15.90 - 46.96$ ms for $w = 1$ bit-sized circuit. For $w = 8$ (i.e., user inputs are bytes), the Delegate costs about $300 - 700$ ms in exchange for faster client-side computation. The primary use case for the proposed scheme is for the delegator to pre-delegate the dmk dormant key during the offline phase. The trade-off of faster client-side computation is favorable for the aforementioned use case. With the configuration of using byte-size circuit $w = 8$, signing redelegation RDelegate using TEE and StrongBox hardware only takes approximately 116.35 ms and 333.9 ms, respectively, because of the smaller ℓ_{tag} size in comparison to ℓ_{ecdsa}. The full signature signing DSign requires computation times of 337 ms for the software-based keystore and 1.6s for the hardware-backed keystore. Activate has a minimal performance impact on the server, with only a total of 2.07 ms required. Lastly, Unlock computation time is included in the RDelegate and DSign operations benchmark.

Communication Analysis. The final signature is a standard ECDSA signature and therefore the size of it remains the same. The delegatee client sends over an attested delegatee public key which is around $|\mathsf{pk}_D| \approx 3.7$ KB. The data sent by the server is dominated by the one-time dormant signing circuit dmk, thus, we focus on the size of the dmk in the remaining section. For partial key dmk consisting only of functionality for RDelegate, the key can be as small as 8 KB or be just about 122 KB depending on the performance tradeoff. Meanwhile, for punctured dmk consisting only DSign functionality, the dmk key is larger, taking around 50 KB for the bit-size case $w = 1$ and taking around 776 KB for the byte-size case $w = 8$. The result is summarized in Table 2 given in the Appendix.

References

[1] Amirhossein Adavoudi Jolfaei, Seyed Farhad Aghili, and Dave Singelee. 2021. A Survey on Blockchain-Based IoMT Systems: Towards Scalability. *IEEE Access* 9 (2021), 148948–148975. doi:10.1109/ACCESS.2021.3117662

[2] Asmaa Aldoubaee, Noor Hassan, and Fiza Abdul Rahim. 2023. A Systematic Review on Blockchain Scalability. *International Journal of Advanced Computer Science and Applications* 14 (09 2023). doi:10.14569/IJACSA.2023.0140981

[3] Ryan Amos, Marios Georgiou, Aggelos Kiayias, and Mark Zhandry. 2020. One-shot signatures and applications to hybrid quantum/classical authentication. In *52nd Annual ACM Symposium on Theory of Computing*, Konstantin Makarychev, Yury Makarychev, Madhur Tulsiani, Gautam Kamath, and Julia Chuzhoy (Eds.). ACM Press, 255–268. doi:10.1145/3357713.3384304

[4] Lukas Aumayr, Oguzhan Ersoy, Andreas Erwig, Sebastian Faust, Kristina Hostáková, Matteo Maffei, Pedro Moreno-Sanchez, and Siavash Riahi. 2021. Generalized Channels from Limited Blockchain Scripts and Adaptor Signatures. In *Advances in Cryptology – ASIACRYPT 2021, Part II (Lecture Notes in Computer Science, Vol. 13091)*, Mehdi Tibouchi and Huaxiong Wang (Eds.). Springer, Cham, 635–664. doi:10.1007/978-3-030-92075-3_22

[5] Lukas Aumayr, Sri Aravinda Krishnan Thyagarajan, Giulio Malavolta, Pedro Moreno-Sanchez, and Matteo Maffei. 2022. Sleepy Channels: Bi-directional Payment Channels without Watchtowers. In *ACM CCS 2022: 29th Conference on Computer and Communications Security*, Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi (Eds.). ACM Press, 179–192. doi:10.1145/3548606.3559370

[6] Paulo S. L. M. Barreto, Ben Lynn, and Michael Scott. 2003. Constructing Elliptic Curves with Prescribed Embedding Degrees. In *SCN 02: 3rd International Conference on Security in Communication Networks (Lecture Notes in Computer Science, Vol. 2576)*, Stelvio Cimato, Clemente Galdi, and Giuseppe Persiano (Eds.). Springer, Berlin, Heidelberg, 257–267. doi:10.1007/3-540-36413-7_19

[7] Paulo S. L. M. Barreto and Michael Naehrig. 2006. Pairing-Friendly Elliptic Curves of Prime Order. In *SAC 2005: 12th Annual International Workshop on Selected Areas in Cryptography (Lecture Notes in Computer Science, Vol. 3897)*, Bart Preneel and Stafford Tavares (Eds.). Springer, Berlin, Heidelberg, 319–331. doi:10.1007/11693383_22

[8] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. 2012. From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again. In *ITCS 2012: 3rd Innovations in Theoretical Computer Science*, Shafi Goldwasser (Ed.). Association for Computing Machinery, 326–349. doi:10.1145/2090236.2090263

[9] Dan Boneh and Matthew K. Franklin. 2001. Identity-Based Encryption from the Weil Pairing. In *Advances in Cryptology – CRYPTO 2001 (Lecture Notes in Computer Science, Vol. 2139)*, Joe Kilian (Ed.). Springer, Berlin, Heidelberg, 213–229. doi:10.1007/3-540-44647-8_13

[10] Raymond Cheng, Fan Zhang, Jernej Kos, Warren He, Nicholas Hynes, Noah M. Johnson, Ari Juels, Andrew Miller, and Dawn Song. 2019. Ekiden: A Platform for Confidentiality-Preserving, Trustworthy, and Performant Smart Contracts. In *2019 IEEE European Symposium on Security and Privacy*. IEEE Computer Society Press, 185–200. doi:10.1109/EuroSP.2019.00023

[11] Hao Chung, Elisaweta Masserova, Elaine Shi, and Sri AravindaKrishnan Thyagarajan. 2022. Rapidash: Improved Constructions for Side-Contract-Resilient Fair Exchange. Cryptology ePrint Archive, Report 2022/1063. https://eprint.iacr.org/2022/1063

[12] Yang Cui, Eiichiro Fujisaki, Goichiro Hanaoka, Hideki Imai, and Rui Zhang. 2007. Formal Security Treatments for Signatures from Identity-Based Encryption. In *ProvSec 2007: 1st International Conference on Provable Security (Lecture Notes in Computer Science, Vol. 4784)*, Willy Susilo, Joseph K. Liu, and Yi Mu (Eds.). Springer, Berlin, Heidelberg, 218–227. doi:10.1007/978-3-540-75670-5_16

[13] Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. 2020. Flash boys 2.0: Frontrunning, transaction reordering, and consensus instability in decentralized exchanges. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 910–927.

[14] Poulami Das, Lisa Eckey, Tommaso Frassetto, David Gens, Kristina Hostáková, Patrick Jauernig, Sebastian Faust, and Ahmad-Reza Sadeghi. 2019. FastKitten: Practical Smart Contracts on Bitcoin. In *USENIX Security 2019: 28th USENIX Security Symposium*, Nadia Heninger and Patrick Traynor (Eds.). USENIX Association, 801–818.

[15] Nico Döttling, Lucjan Hanzlik, Bernardo Magri, and Stella Wohnig. 2023. McFly: Verifiable Encryption to the Future Made Practical. In *FC 2023: 27th International Conference on Financial Cryptography and Data Security, Part I (Lecture Notes in Computer Science, Vol. 13950)*, Foteini Baldimtsi and Christian Cachin (Eds.). Springer, Cham, 252–269. doi:10.1007/978-3-031-47754-6_15

[16] Paul Feldman. 1987. A Practical Scheme for Non-interactive Verifiable Secret Sharing. In *28th Annual Symposium on Foundations of Computer Science*. IEEE Computer Society Press, 427–437. doi:10.1109/SFCS.1987.4

[17] Ethereum Foundation. 2023. Ethereum 2.0: Scaling Ethereum via Sharding. https://ethereum.org/en/roadmap/sharding/.

[18] Ethereum Foundation. 2023. Ethereum Scalability and TPS Limitations. https://ethereum.org/en/developers/docs/scalability/.

[19] Ethereum Foundation. 2023. zk-Rollups: Scaling Ethereum with Zero-Knowledge Proofs. https://ethereum.org/en/developers/docs/scaling/zk-rollups/.

[20] Jens Groth. 2016. On the Size of Pairing-Based Non-interactive Arguments. In *Advances in Cryptology – EUROCRYPT 2016, Part II (Lecture Notes in Computer Science, Vol. 9666)*, Marc Fischlin and Jean-Sébastien Coron (Eds.). Springer, Berlin, Heidelberg, 305–326. doi:10.1007/978-3-662-49896-5_11

[21] Jens Groth and Victor Shoup. 2022. On the Security of ECDSA with Additive Key Derivation and Presignatures. In *Advances in Cryptology – EUROCRYPT 2022, Part I (Lecture Notes in Computer Science, Vol. 13275)*, Orr Dunkelman and Stefan Dziembowski (Eds.). Springer, Cham, 365–396. doi:10.1007/978-3-031-06944-4_13

[22] Lucjan Hanzlik, Julian Loss, Sri Aravinda Krishnan Thyagarajan, and Benedikt Wagner. 2024. Sweep-UC: Swapping Coins Privately. In *2024 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 3822–3839. doi:10.1109/SP54263.2024.00081

[23] Dennis Hofheinz, Tibor Jager, and Edward Knapp. 2012. Waters Signatures with Optimal Security Reduction. In *PKC 2012: 15th International Conference on Theory and Practice of Public Key Cryptography (Lecture Notes in Computer Science, Vol. 7293)*, Marc Fischlin, Johannes Buchmann, and Mark Manulis (Eds.). Springer, Berlin, Heidelberg, 66–83. doi:10.1007/978-3-642-30057-8_5

[24] Visa Inc. 2023. Visa Acceptance for Retailers. https://usa.visa.com/run-your-business/small-business-tools/retail.html.

[25] Stephan Krenn and Thomas Lorünser. 2021. Single-Use Delegatable Signatures Based on Smart Contracts. In *Proceedings of the 16th International Conference on Availability, Reliability and Security* (Vienna, Austria) *(ARES '21)*. Association for Computing Machinery, New York, NY, USA, Article 40, 7 pages. doi:10.1145/3465481.3469192

[26] Joshua Lind, Oded Naor, Ittay Eyal, Florian Kelbert, Emin Gün Sirer, and Peter Pietzuch. 2019. Teechain: a secure payment network with asynchronous blockchain access. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles* (Huntsville, Ontario, Canada) *(SOSP '19)*. Association for Computing Machinery, New York, NY, USA, 63–79. doi:10.1145/3341301.3359627

[27] Varun Madathil, Sri Aravinda Krishnan Thyagarajan, Dimitrios Vasilopoulos, Lloyd Fournier, Giulio Malavolta, and Pedro Moreno-Sanchez. 2023. Cryptographic Oracle-based Conditional Payments. In *ISOC Network and Distributed System Security Symposium – NDSS 2023*. The Internet Society.

[28] Ian Miers, Christina Garman, Matthew Green, and Aviel D. Rubin. 2013. Zerocoin: Anonymous Distributed E-Cash from Bitcoin. In *2013 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 397–411. doi:10.1109/SP.2013.34

[29] Andrew Miller, Iddo Bentov, Ranjit Kumaresan, and Patrick McCorry. 2019. Sprites and State Channels: Payment Networks That Go Faster Than Lightning. In *International Conference on Financial Cryptography and Data Security*. Springer, 508–526.

[30] Satoshi Nakamoto. 2008. Bitcoin: A peer-to-peer electronic cash system. (2008).

[31] Muhammad Hassan Nasir, Junaid Arshad, Muhammad Mubashir Khan, Mahawish Fatima, Khaled Salah, and Raja Jayaraman. 2022. Scalable blockchains — A systematic review. *Future Gener. Comput. Syst.* 126, C (Jan. 2022), 136–162. doi:10.1016/j.future.2021.07.035

[32] Joseph Poon and Thaddeus Dryja. 2016. The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments. In *White Paper*. https://lightning.network/lightning-network-paper.pdf.

[33] NEAR Protocol. 2023. Nightshade: Near Protocol's Approach to Sharding. https://near.org/blog/near-protocol-nightshade-sharding/.

[34] Gabriel Antonio F. Rebello, Gustavo F. Camilo, Lucas Airam C. de Souza, Maria Potop-Butucaru, Marcelo Dias de Amorim, Miguel Elias M. Campista, and Luís Henrique M. K. Costa. 2024. A Survey on Blockchain Scalability: From Hardware to Layer-Two Protocols. *IEEE Communications Surveys and Tutorials* 26, 4 (2024), 2411–2458. doi:10.1109/COMST.2024.3376252

[35] Flashbots Research. 2023. MEV-Boost: Enhancing Ethereum's Block Production. https://boost.flashbots.net/.

[36] Luis F. G. Sarmenta, Marten van Dijk, Charles W. O'Donnell, Jonathan Rhodes, and Srinivas Devadas. 2006. Virtual monotonic counters and count-limited objects using a TPM without a trusted OS. In *Proceedings of the First ACM Workshop on Scalable Trusted Computing* (Alexandria, Virginia, USA) *(STC '06)*. Association for Computing Machinery, New York, NY, USA, 27–42. doi:10.1145/1179474.1179485

[37] Srinath Setty. 2020. Spartan: Efficient and General-Purpose zkSNARKs Without Trusted Setup. In *Advances in Cryptology – CRYPTO 2020, Part III (Lecture Notes in Computer Science, Vol. 12172)*, Daniele Micciancio and Thomas Ristenpart (Eds.). Springer, Cham, 704–737. doi:10.1007/978-3-030-56877-1_25

[38] Ertem Nusret Tas, István András Seres, Yinuo Zhang, Márk Melczer, Mahimna Kelkar, Joseph Bonneau, and Valeria Nikolaenko. 2024. Atomic and Fair Data Exchange via Blockchain. In *ACM CCS 2024: 31st Conference on Computer and Communications Security*, Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie (Eds.). ACM Press, 3227–3241. doi:10.1145/3658644.3690248

[39] Optimism Team. 2023. Optimism: Scaling Ethereum with Optimistic Rollups. https://www.optimism.io/.

[40] Sri Aravinda Krishnan Thyagarajan and Giulio Malavolta. 2021. Lockable Signatures for Blockchains: Scriptless Scripts for All Signatures. In *2021 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 937–954.

doi:10.1109/SP40001.2021.00065
[41] Sri Aravinda Krishnan Thyagarajan, Giulio Malavolta, Fritz Schmid, and Dominique Schröder. 2022. Verifiable Timed Linkable Ring Signatures for Scalable Payments for Monero. In *ESORICS 2022: 27th European Symposium on Research in Computer Security, Part II (Lecture Notes in Computer Science, Vol. 13555)*, Vijayalakshmi Atluri, Roberto Di Pietro, Christian Damsgaard Jensen, and Weizhi Meng (Eds.). Springer, Cham, 467–486. doi:10.1007/978-3-031-17146-8_23
[42] Sarisht Wadhwa, Jannis Stoeter, Fan Zhang, and Kartik Nayak. 2023. He-HTLC: Revisiting Incentives in HTLC. In *ISOC Network and Distributed System Security Symposium – NDSS 2023*. The Internet Society.
[43] Brent R. Waters. 2005. Efficient Identity-Based Encryption Without Random Oracles. In *Advances in Cryptology – EUROCRYPT 2005 (Lecture Notes in Computer Science, Vol. 3494)*, Ronald Cramer (Ed.). Springer, Berlin, Heidelberg, 114–127. doi:10.1007/11426639_7
[44] Gavin Wood. 2016. Polkadot: Vision for a heterogeneous multi-chain framework. In *White Paper*. https://polkadot.network/PolkaDotPaper.pdf.
[45] Wei-Zhu Yeoh, Lucjan Hanzlik, and Oliver Valta. 2024. Delegating FIDO Credentials Using Single-use ECDSA Signatures. In *ASIACCS 24: 19th ACM Symposium on Information, Computer and Communications Security*, Jianying Zhou, Tony Q. S. Quek, Debin Gao, and Alvaro A. Cárdenas (Eds.). ACM Press. doi:10.1145/3634737.3657004

A Additional Background

We recall all other needed background in this section.

A.1 Digital Signatures.

A digital signature scheme DS, has a key generation algorithm $\mathsf{KGen}(\lambda)$ that takes the security parameter λ and outputs the public/secret key pair $(\mathsf{pk}, \mathsf{sk})$, a signing algorithm $\mathsf{Sign}(\mathsf{sk}, m)$ that inputs sk and a message $m \in \{0,1\}^*$ and outputs a signature σ, and a verification algorithm $\mathsf{Verify}(\mathsf{pk}, \sigma, \mathsf{m})$ that outputs 1 if σ is a valid signature on m under pk, and outputs 0 otherwise. A signature scheme is existentially unforgeable under a chosen-message attack (EUF-CMA) if, for all PPT adversary $\mathcal{A}$, it cannot generate a new signature even after having seen many valid adversarially chosen message-signature pairs.

Definition A.1. *A signature scheme is a tuple of algorithms* DS :=(KeyGen,Sign,Verify*) defined as follows:*

$\underline{\mathsf{KeyGen}(\lambda)}$: *on input a security parameter λ, this algorithm outputs a signing key* sk *and a verification key* vk.

$\underline{\mathsf{Sign}(\mathsf{sk}, m)}$: *on input a signing key* sk *and a message m, this algorithm outputs a signature σ.*

$\underline{\mathsf{Verify}(\mathsf{vk}, \sigma, \mathsf{m})}$: *on input a verification key* vk*, a signature σ and a message m, this algorithm outputs 1 if σ is valid and 0 otherwise.*

Informally, a signature scheme is correct if for all security parameter λ, all key pairs $(\mathsf{vk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(\lambda)$, it holds that $\mathsf{Verify}(\mathsf{vk}, \mathsf{Sign}(\mathsf{sk}, \mathsf{m}), \mathsf{m}) = 1$, for every message m.

Definition A.2. *The ECDSA signature scheme consists of the following algorithms:*

$\underline{\mathsf{KeyGen}_{\mathsf{ECDSA}}(\lambda)}$: *on input a security parameter λ, this algorithm outputs signing key* $\mathsf{sk} \leftarrow\!\!\$\ \mathbb{Z}_q^*$ *and verification key* $\mathsf{vk} := g^{\mathsf{sk}}$.

$\underline{\mathsf{Sign}_{\mathsf{ECDSA}}(\mathsf{sk}, m)}$: *on input a signing key* sk *and a message m, this algorithm samples a uniform $k \in \mathbb{Z}_q^*$ and sets $r := F(g^k)$ where $F((x,y)) \stackrel{def}{=} x \mod q$. Then, it computes $s \leftarrow k^{-1} \cdot (\mathsf{H}(m) + \mathsf{sk} \cdot r) \mod q$. The algorithm outputs a signature $\sigma := (r, s)$.*

$\underline{\mathsf{Verify}_{\mathsf{ECDSA}}(\mathsf{vk}, \sigma, \mathsf{m})}$: *on input a verification key* vk*, a signature σ and a message m, this algorithm parses $(r, s) := \sigma$ with $r, s \neq 0 \mod q$ and outputs 1 if and only if $r = F(k')$, where $k' := (g^{\mathsf{H}(m)} \cdot (\mathsf{vk})^r)^{s^{-1}}$.*

A.2 nUKE Security

Definition A.3 (n-Use Real-or-Random Key Security). *We say that an* nUKE *scheme is RoRL secure if it holds that every PPT adversary $\mathcal{A}$ has at most negligible advantage in the following experiment, where the advantage is defined as* $\mathbf{Adv}_{\mathsf{RoRL}}(\mathcal{A}) := |\Pr[\mathsf{RoRL}_{\mathcal{A}}(\lambda) = 1] - 1/2|$.

$\mathsf{RoRL}_{\mathcal{A}}(\lambda)$

$(\mathsf{gk}_{\mathsf{Att}}, \mathsf{vk}_{\mathsf{Att}}) \leftarrow \mathsf{nUKE.Setup}_{\mathsf{Att}}(\lambda)$
$\mathcal{O} := \{O_1(\mathsf{gk}_{\mathsf{Att}}), O_2(\cdot,\cdot)\}$
$(\mathsf{st}, (\mathsf{pk}_A, \sigma_{\mathsf{att}})) \leftarrow \mathcal{A}^{\mathcal{O}}(\mathsf{vk}_{\mathsf{Att}})$
abort if $\mathsf{nUKE.Verify}_{\mathsf{Att}}(\mathsf{pk}_A, n, \mathsf{vk}_{\mathsf{Att}}, \sigma_{\mathsf{att}}) = 0$
return 1 *if* $(\cdot, \mathsf{pk}_A, \cdot, \ell) \notin \mathcal{KP}_{\mathsf{att}} \ \wedge$
$\mathsf{Verify}_{\mathsf{Att}}(\mathsf{pk}_A, n, \mathsf{nUKE.vk}_{\mathsf{Att}}, \sigma_{\mathsf{att}}) = 1$
for $i \in [n+1]$ *then*
$b_i \leftarrow\!\!\$\ \{0,1\}$; $(\mathsf{sk}_i, \mathsf{pk}_i) \leftarrow \mathsf{nUKE.KeyGen}(\lambda)$
if b_i *then* $K_i \leftarrow\!\!\$\ \mathcal{K}$
else $K_i \leftarrow \mathsf{nUKE.Shared}(\mathsf{sk}_i, \mathsf{pk}_A)$
$d \leftarrow \mathcal{A}^{\mathcal{O}}(\mathsf{st}, \{(\mathsf{pk}_i, K_i)\}_{i \in [n+1]})$
return $d = \oplus_{i \in [n+1]} b_i$

$O_1(\mathsf{gk}_{\mathsf{Att}})$

$(\mathsf{sk}, \mathsf{pk}, \sigma_{\mathsf{att}}) \leftarrow \mathsf{nUKE.KeyGen}_{\mathsf{Att}}(\mathsf{gk}_{\mathsf{Att}}, n)$
$\mathcal{KP}_{\mathsf{att}} := \mathcal{KP}_{\mathsf{att}} \cup \{(\mathsf{sk}, \mathsf{pk}, \sigma_{\mathsf{att}}, \ell)\}$
return $(\mathsf{pk}, \sigma_{\mathsf{att}})$

$O_2(\mathsf{pk}_A, \mathsf{pk}_C)$

if $(\mathsf{sk}_A, \mathsf{pk}_A, \sigma_{\mathsf{att}}, \ell) \notin \mathcal{KP}_{\mathsf{att}}$ *then abort*
if $\ell < 1$ *then abort*
$\mathcal{KP}_{\mathsf{att}} := \mathcal{KP}_{\mathsf{att}} \,/\, \{(\mathsf{sk}_A, \mathsf{pk}_A, \sigma_{\mathsf{att}}, \ell)\}$
$\mathcal{KP}_{\mathsf{att}} := \mathcal{KP}_{\mathsf{att}} \cup \{(\mathsf{sk}_A, \mathsf{pk}_A, \sigma_{\mathsf{att}}, \ell - 1)\}$
$K \leftarrow \mathsf{nUKE.Shared}(\mathsf{sk}_A, \mathsf{pk}_C)$
return K

A.3 Identity-based Encryption

Definition A.4 (IND-ID-CPA Security). *We say that an* IBE *scheme is* IND-ID-CPA *secure if it holds that every PPT adversary $\mathcal{A}$ has at most negligible advantage in the following experiment, where the advantage is defined as* $\mathbf{Adv}_{\mathsf{IND\text{-}ID\text{-}CPA}}(\mathcal{A}) := |\Pr[\mathsf{IND\text{-}ID\text{-}CPA}_{\mathcal{A}}(\lambda) = 1] - 1/2|$.

Operation	Size	Size* (KB)	
		$w = 1$	$w = 8$
RDelegate	$\frac{\ell_{\text{tag}}}{w} \cdot 2^w(\lvert\mathbb{G}_{KE}\rvert + \lvert\mathbb{C}_{\mathsf{KE}}\rvert + \lvert\mathbb{C}_{d'}\rvert) + 2\lvert\mathbb{G}_1\rvert + \lvert\mathbb{G}_2\rvert + \lvert\mathbb{C}_{\mathsf{IBE}}\rvert$	8.17	121.84
DSign	$\frac{\ell_{\text{ECDSA}}}{w} \cdot 2^w(\lvert\mathbb{G}_{KE}\rvert + \lvert\mathbb{C}_{\mathsf{KE}}\rvert + \lvert\mathbb{C}_{d}\rvert) + 2\lvert\mathbb{Z}_q^*\rvert + \lvert\mathbb{C}_{\mathsf{IBE}}\rvert$	49.03	776.53

* where $\ell_{\text{tag}} = 5$B; $\ell_{\text{ECDSA}} = 32$B; $\lvert\mathbb{G}_{KE}\rvert = 33$B; $\lvert\mathbb{C}_{\mathsf{KE}}\rvert, \lvert\mathbb{C}_{d}\rvert, \lvert\mathbb{C}_{d'}\rvert, \lvert\mathbb{G}_1\rvert, \lvert\mathbb{Z}_q^*\rvert = 32$B; $\lvert\mathbb{G}_2\rvert = 64$B; $\lvert\mathbb{C}_{\mathsf{IBE}}\rvert = 480$B.

Table 2: Circuit Size Overview for ORBIT.

$\text{IND-ID-CPA}_{\mathcal{A}}(\lambda)$	$O_K(\mathsf{sk}, \mathsf{ID})$
$(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{IBE.Setup}(\lambda)$	$\mathsf{dk}_{\mathsf{ID}} \leftarrow \mathsf{IBE.KeyDer}(\mathsf{sk}, \mathsf{ID})$
$O := \{O_K(\mathsf{sk}, \cdot))\}$	$\mathcal{K}_{\mathsf{ID}} := \mathcal{K}_{\mathsf{ID}} \cup \mathsf{dk}_{\mathsf{ID}}$
$(\mathsf{st}, id, m_0, m_1) \leftarrow \mathcal{A}^O(\mathsf{pk})$	*return* $\mathsf{dk}_{\mathsf{ID}}$
$b \leftarrow\$ \{0, 1\}$	
$c \leftarrow \mathsf{Encrypt}(\mathsf{pk}, id, m_b)$	
$(\mathsf{st}, b') \leftarrow \mathcal{A}^O(c)$	
abort if $id \in \mathcal{K}_{\mathsf{ID}}$	
return $b = b'$	

A.4 Non-intereactive Zero-knowledge

Let $\mathcal{R}$ be an relation and $\mathcal{L}_{\mathcal{R}}$ be the language defined by $\mathcal{R}$. A non-interactive zero-knowledge argument NIZK = (NIZK.Setup, NIZK.Prove, NIZK.Verify) for $\mathcal{L}_{\mathcal{R}}$ consists of PPT algorithms:

- $\underline{\mathsf{NIZK.Setup}(\lambda)}$: This algorithm on input security parameter λ, outputs a common reference string crs.
- $\underline{\mathsf{NIZK.Prove}(\mathsf{crs}, x, w)}$: This algorithm on input a common reference string crs, a statement x and a witness w, outputs either a proof π or $\perp$.
- $\underline{\mathsf{NIZK.Verify}(\mathsf{crs}, \mathsf{x}, \pi)}$: This algorithm on input a common reference string crs, a statement x and a proof π, outputs 1 if π is a valid proof and 0 otherwise.

Definition A.5 (NIZK Property).

Computational Soundness: *We define the advantage* $\mathsf{Sound}^{\mathcal{A}}(\lambda)$ *of an adversary* $\mathcal{A}$ *in breaking the soundness of the proof system as the probability defined below, where the probability is taken over the random coins of* NIZK.Verify.

$$\Pr\left[\begin{array}{c} \mathsf{NIZK.Verify}(\mathsf{crs}, \mathsf{stmt}, \pi_{\mathsf{NIZK}}) = 1 \ : \\ \mathsf{crs} \leftarrow \mathsf{NIZK.Setup}(\lambda); \\ (\pi_{\mathsf{NIZK}}, \mathsf{stmt}) \leftarrow \mathcal{A}^{\mathsf{Verify}(\mathsf{crs}, \cdot, \cdot)}(\mathsf{crs}); \\ \mathsf{stmt} \notin \mathcal{L}_{\mathcal{R}} \end{array}\right]$$

Zero-Knowledge: *We define the advantage* $\mathsf{ZK}^{\mathcal{A}}(\lambda)$ *of an adversary* $\mathcal{A}$ *in breaking the zero-knowledge property of the proof system as the probability defined below, where the probability is taken over random coins of* NIZK.Prove. *We say that the proof system is zero-knowledge if there exist simulator algorithms* $(\mathcal{SIM}_1, \mathcal{SIM}_2)$ *such that for all PPT adversaries* $\mathcal{A}$ *there exists a negligible function* negl *such that* $\mathsf{ZK}^{\mathcal{A}}(\lambda) \leq \mathsf{negl}$.

$$\left|\Pr\left[\begin{array}{c} \mathcal{A}(\pi^*_{\mathsf{NIZK}}) = 1 \ : \\ \end{array}\begin{array}{c} \mathsf{crs} \leftarrow \mathsf{NIZK.Setup}(\lambda); \\ (\mathsf{stmt}, \mathsf{wit}) \leftarrow \mathcal{A}(\mathsf{crs}); \\ \mathcal{R}(\mathsf{stmt}, \mathsf{wit}) = 1; \\ \pi^*_{\mathsf{NIZK}} \leftarrow \mathsf{NIZK.Prove}(\mathsf{crs}, \\ \mathsf{stmt}, \mathsf{wit}) \end{array}\right] - \Pr\left[\begin{array}{c} \mathcal{A}(\pi^*_{\mathsf{NIZK}}) = 1 \ : \\ \end{array}\begin{array}{c} (\mathsf{crs}, \mathsf{st}) \leftarrow \mathcal{SIM}_1(\lambda); \\ (\mathsf{stmt}, \mathsf{wit}) \leftarrow \mathcal{A}(\mathsf{crs}); \\ \mathcal{R}(\mathsf{stmt}, \mathsf{wit}) = 1; \\ \pi^*_{\mathsf{NIZK}} \leftarrow \mathcal{SIM}_2(\mathsf{crs}, \mathsf{stmt}, \mathsf{st}) \end{array}\right]\right|$$

B Deferred Security Analysis

Proof (of Lemma 5.1). The correctness of the underlying digital signature scheme follows from the ECDSA signature scheme as no modification is made to it. We first show that given a correct activation key $\mathsf{ak}_{\mathsf{tag}}$ for tag and $\mathsf{dmk}_{\mathsf{tag}}$, we can obtain the next correct activation key $\mathsf{ak}_{\mathsf{tag}'}$. The correctness of Waters IBE scheme ensures that given a correct decryption key $\mathsf{ak}_{\mathsf{tag}}$, the Waters IBE ciphertext c_{ak} can be decrypted correctly which transforms $\mathsf{dmk}_{\mathsf{tag}}$ into $\mathsf{osk}_{\mathsf{tag}}$ in $\mathsf{Activate}(\mathsf{dmk}_{\mathsf{tag}}, \mathsf{ak}_{\mathsf{tag}}, \mathsf{pk}, \mathsf{tag})$. Then during the computation of $\mathsf{RDelegate}(\mathsf{osk}, \mathsf{sk}_D, \mathsf{tag})$, as nUKE is correct, the computed shared keys $\{K_{i,\mathsf{btag}_i}\}_{i\in[\ell_m]}$ are the same with the one used to encrypt the payload. It follows that all values $(a_i||b_{i,\mathsf{btag}_i})$ and $a_{\mathcal{S},1}$ can be computed correctly from their ciphertext, which is then used to correctly compute $a := (\oplus_{i\in[\ell_m]} a_i) \oplus a_{\mathsf{tag}}$. With the correctly reconstructed a, $\mathsf{dk}_2 := \mathsf{H}_2(a||1) \oplus c_{\gamma,1}$,$\mathsf{dk}_3 := \mathsf{H}_2(a||2) \oplus c_{\gamma,2}$ can be correctly computed. Let $\{\mathsf{btag}_1, \ldots, \mathsf{btag}_{\ell_m}\}$ be the bit decomposition of tag′. By computing $\mathsf{H}_2(a||b_{i,\mathsf{btag}_i}) \oplus d'_{i,\mathsf{btag}_i}$, for each $i \in [\ell_m]$, the delegatee will receive p'_i corresponding to $(u_i^{\gamma})^{\mathsf{btag}_i} \cdot t'_i$. Now, the computed value dk_1 is a valid component of Waters decryption key $\mathsf{dk}_{\mathsf{tag}'} := (\mathsf{dk}_1, \mathsf{dk}_2, \mathsf{dk}_3)$ as needed by activation key $\mathsf{ak}_{\mathsf{tag}'}$, since:

$$\begin{aligned} \mathsf{dk}_1 = p'_0 \cdot \prod_{i\in[\ell_m]} p'_i &= (\mathsf{sk}_{\mathsf{Wat}} \cdot u_0^{\gamma} \cdot t'_0) \cdot \prod_{i\in[\ell_m]} \left(u_i^{\gamma \cdot \mathsf{btag}_i} \cdot t'_i\right) \\ &= \mathsf{sk}_{\mathsf{Wat}} \cdot (u_0 \prod_{i=1}^{\ell_m} u_i^{\mathsf{btag}_i})^{\gamma} \\ &\quad \cdot \prod_{i\in[\ell_m]} t_i'^{(-1)} \cdot \prod_{i\in[\ell_m]} t'_i \\ &= \mathsf{sk}_{\mathsf{Wat}} \cdot (u_0 \prod_{i=1}^{\ell_m} u_i^{\mathsf{btag}_i})^{\gamma}. \end{aligned}$$

The first activation key $\mathsf{ak}_{\mathsf{tag},1} := \mathsf{dk}_{\mathsf{tag},1}$ which is a decryption key for public key $pk_{\mathsf{Wat}_{\mathsf{tag},1}}$ is computed by Waters key derivation algorithm $\mathsf{KeyDer}_{\mathsf{Wat}}$ in $\mathsf{Unlock}(\mathsf{sk}, \mathsf{tag})$. Based on the correctness

of redelegate, applying the redelegate iteratively to the delegation chain will result in a corerct ak_n and consequently osk_n.

We now show that delegated signing DSign with a valid osk_n will produce a valid ECDSA signature for message m. By applying the analogous procedure above, the delegatee will receive p_i which will correspond to $h_i \cdot 2^{i-1} \cdot k^{-1} + t_i$ together with p_0 and r. Finally, the computed value s is a valid component of an ECDSA signature $\sigma = (r, s)$, since:

$$\begin{aligned} s = p_0 + \sum_{i\in[\ell_m]} p_i &= (k^{-1} \cdot r \cdot \mathsf{sk} + t_0) + \sum_{i\in[\ell_m]} (h_i \cdot 2^{i-1} \cdot k^{-1} + t_i) \\ &= k^{-1}(r \cdot \mathsf{sk} + \sum_{i\in[\ell_m]} (h_i \cdot 2^{i-1})) \\ &\quad + \sum_{i\in[\ell_m]} (-1) \cdot t_i + \sum_{i\in[\ell_m]} t_i \\ &= k^{-1}(\mathsf{H}(m) + r \cdot \mathsf{sk}). \end{aligned}$$

□

Proof (of Theorem 5.2). We will show this theorem via a series of small changes we make to experiment.

Game $\mathbf{G}_0$: The original one-timeness experiment $\Pr[\mathrm{G}_0(\mathcal{A}) = 1] := \mathbf{Adv}_{\mathsf{OTI}}(\mathcal{A})$.

Game $\mathbf{G}_1$: We abort the game if there is a collision in the responses in any of the random oracles H_t where $t \in \{1, 2\}$.

Claim: We claim that the adversary's $\mathcal{A}$ advantage in G_0 and G_1 only differs by a negligible factor:

$$|\Pr[\mathrm{G}_1(\mathcal{A}) = 1] - \Pr[\mathrm{G}_0(\mathcal{A}) = 1]| \leq \frac{Q^2_{\mathsf{H}_1}}{2^{\ell_m}} + \frac{Q^2_{\mathsf{H}_2}}{2^n}$$

We apply a union bound over all $Q^2_{\mathsf{H}_t}$ pairs of random oracle queries and claim follows.

Game $\mathbf{G}_2$: Let Q_D denote the upper bound on the number of queries the adversary can make to oracle O_D. The experiment picks indexes $i^* \in [Q_D]$ and $j^* \in [\ell_m + 1]$ and aborts the game in the case that for any one-time signing key osk generated by the experiment (i.e., inside O_D), there are queries for both $K_{j^*,0}$ and $K_{j^*,1}$. At the same time, the adversary query using the shared secret a, meaning the adversary decrypted shares for both bits.

Claim: We claim that the following holds:

$$|\Pr[\mathrm{G}_2(\mathcal{A}) = 1] - \Pr[\mathrm{G}_1(\mathcal{A}) = 1]| \leq (Q_D \cdot \ell_m + 2) \cdot \mathbf{Adv}_{\mathsf{RoRL}}(\mathcal{R}_1).$$

Proof. The proof follows identically as in [45] but is included for completeness. We show this proof by creating a reduction algorithm $\mathcal{R}_1$ against the real-or-random security of the nUKE scheme. The reduction sets up the system as described, except for setting the attestation verification key $\mathsf{vk}_{\mathsf{Att}}$ to the one received from the challenger. Each time the adversary $\mathcal{A}$ runs the key generation algorithm, the reduction $\mathcal{R}_1$ queries the O_1 oracle, receiving $(\mathsf{pk}, \mathsf{vk}_{\mathsf{Att}})$ and providing those inside the hardware component to $\mathcal{A}$.

For any $O_D(\mathsf{sk}, \mathsf{pk}_D = \mathsf{pk}_A, \sigma_{\mathsf{att}})$ query made by the adversary, the reduction outputs $(\mathsf{pk}_A, \sigma_{\mathsf{att}})$ if pk_D was not created as part of the simulation. Note that such a pair would immediately break the real-or-random security of the nUKE scheme. In particular, this means that the adversary can break real-or-random security by attesting an arbitrary public key that was not necessarily honestly generated and can potentially be used unlimitedly by $\mathcal{A}$.

If the above case does not happen, the reduction $\mathcal{R}_1$ works as follows. It picks the indexes i^*, j^* and aborts the experiment as in the game's description. For the experiment itself, the reduction proceeds as follows. All calls to O_D are now simulated honestly, except the i^*-th query. Let $\mathsf{pk}^*_D = (\mathsf{pk}^*_A, \sigma^*_{\mathsf{att}})$ be the public key input by $\mathcal{A}$ in that query and ℓ_c be the nUKE counter limit ($\ell_m + 1$). The reduction $\mathcal{R}_1$ returns $(\mathsf{pk}^*_A, \sigma^*_{\mathsf{att}})$ and the real-or-random experiment proceeds to the second phase, while $\mathcal{R}_1$ is given $(\mathsf{pk}_i, K_i)_{i\in[\ell_c+1]}$ as its challenge. The reduction uses its O_2 to get key K'_i for all pk_i for $i \in [\ell_c]$. To win the real-or-random security experiment $\mathcal{R}_1$ only needs to distinguish if K_{ℓ_c+1} is $\mathsf{Shared}(\mathsf{sk}_{\ell_c+1}, \mathsf{pk}_A)$ or random. Note that all the other keys K'_i are valid shared keys.

Now, to answer O_D queries, the reduction computes the one-time signing key osk as follows. It computes all the values honestly except $\mathsf{pk}_{j^*,1}$, which is set to pk_{ℓ_c+1}. Similarly, the reduction uses K_{ℓ_c+1} to compute $c_{j^*,1}$. Assuming the key K_{ℓ_c+1} is real, the reduction will notice it appear as a query to H since this is the condition we abort. Otherwise, if the key is random and the experiment simulation is not perfect, the random key K_{ℓ_c+1} will not appear as input to the random oracle H. In both cases, the reduction will be able to use the adversary $\mathcal{A}$ to distinguish if K_{ℓ_c+1} is a real or a random key and, in the end, return a valid bit d, breaking the real-or-random security of the nUKE scheme, which proves the claim. □

Game $\mathbf{G}_3$: Let Q_D be the upper bound on the number of queries to oracle O_D. The experiment picks an index $t^* \in [Q_D]$. We abort the game when there is a random oracle query using $r_{\mathsf{tag}_{t^*}}$, but there is no query to Unlock oracle O_A using tag_{t^*}, and there are no queries $\mathsf{H}_2(a^t || b^t_{i,\mathsf{btag}^t_i})$ for which $(\mathsf{btag}^t_1, \ldots, \mathsf{btag}^t_{\ell_m})$ are the bit decomposition of tag_{t^*} for all $t \in [Q_D]$. In short, the adversary recovered the r_{tag} from the IBE encrypted ciphertext without the corresponding decryption key.

Claim: We claim the following holds:

$$|\Pr[\mathrm{G}_3(\mathcal{A}) = 1] - \Pr[\mathrm{G}_2(\mathcal{A}) = 1]| \leq Q_D \cdot \mathbf{Adv}_{\mathsf{IND\text{-}ID\text{-}CPA}}(\mathcal{R}_2).$$

Proof. We prove the claim by constructing a reduction algorithm $\mathcal{R}_2$ against the IND-ID-CPA security of IBE scheme. It obtains the $\mathsf{pk}_{\mathsf{IBE}}$ from the challenger and set it as the public key for the Waters IBE scheme. The reduction now simulates the experiment without IBE master secret key. The reduction sets up the system as described, except for garbling of Waters secret. The reduction answers the call to $O_A(\mathsf{ID})$ from the adversary by querying the key derivation oracle for the same id $O_K(\mathsf{ID})$. The reduction computes the Waters garbled terms $p'_0, p'_{i,0}, p'_{i,1}, c_\gamma$ as usual, except all $\mathsf{sk}_{\mathsf{Wat}}$, u^γ_0, u^γ_i, g^γ_1 and g^γ_2 are replaced with random variables $v_{\mathsf{sk}}, v_{u_0}, v_{u_i}, v_{g_1}, v_{g_2}$ respectively. After all $\mathsf{H}_2(K_{i,\mathsf{btag}_i})$ have been queried by the adversary that determined the tag $\mathsf{tag} := (\mathsf{btag}_1, \ldots, \mathsf{btag}_{\ell_m})$, the reduction queries the key derivation oracle for the tag decryption key $(\mathsf{dk}_1, \mathsf{dk}_2, \mathsf{dk}_3) \leftarrow O_K(\mathsf{tag})$. The reduction answers the queries normally except $\mathsf{H}_2(a||b_{1,\mathsf{btag}_1}) := \mathsf{H}_2(a||b_{1,\mathsf{btag}_1}) \oplus p'_{1,\mathsf{btag}_1} \oplus \left(t'_1 \cdot\right.$

$(v_{\mathsf{sk}} \cdot v_{u_0} \cdot \prod_{i=2}^{\ell_m} v_{u_i}^{\mathsf{btag}_i})^{-1} \cdot \mathsf{dk}_1\Big)$, $\mathsf{H}_2(a||1) := \mathsf{H}_2(a||1) \oplus v_{g_1} \oplus \mathsf{dk}_2$ and $\mathsf{H}_2(a||2) := \mathsf{H}_2(a||2) \oplus v_{g_2} \oplus \mathsf{dk}_3$.

The reduction picks an index $t^* \in [Q_D]$. The reduction submits $m_0 := r_{\mathsf{tag}_{t^*}}$ and $m_1 := r'_{\mathsf{tag}_{t^*}}$ as the message and receive c_b. The reduction set $c_{\mathsf{ak}_{t^*}} := c_b$. If the abort condition occur for either $r_{\mathsf{tag}_{t^*}}$ or $r'_{\mathsf{tag}_{t^*}}$, then the reduction returns $b := 0$ if $r_{\mathsf{tag}_{t^*}}$ is queried or $b := 1$ if $r'_{\mathsf{tag}_{t^*}}$ is queried.

□

Note that at this point because of abort condition in G_2, the adversary can only irreversibly compute at most one ECDSA signature or Waters decryption key exclusively from one activated one-time signing key osk. Any activated one-time signing key osk is traceable back to at minimum one genesis activation key issued by Unlock because of abort condition introduced in G_3. Combining these conditions, we have that total number of ECDSA signature computed from DSign ℓ_{DSign} must be less than or equal to the total number of Unlock query ℓ, i.e. $\ell_{\mathsf{DSign}} \leq \ell$. We now show that $\Pr[G_3(\mathcal{A}) = 1] \leq \mathbf{Adv}_{\mathsf{UNF}}(\mathcal{R}_3)$. It means that we can bind the probability of the adversary winning in G_3, by the probability of reduction $\mathcal{R}_3$ breaking the existential unforgeability of the ECDSA signature scheme with "raw" queries.

The reduction receives the ECDSA verification key vk from its challenger and provides it to the adversary $\mathcal{A}$. $\mathcal{R}_3$ simulates the experiments perfectly except for O_D. For any query with $\mathsf{pk}_D = (\mathsf{pk}_A, \sigma_{\mathsf{att}})$ to this oracle, it proceeds as follows. For each $i \in [\ell_m]$ it picks random keys $(\mathsf{sk}_{i,0}, \mathsf{pk}_{i,0}) \leftarrow\!\$\ \mathsf{nUKE.KeyGen}(\lambda)$ and $(\mathsf{sk}_{i,1}, \mathsf{pk}_{i,1}) \leftarrow\!\$\ \mathsf{nUKE.KeyGen}(\lambda)$. It also picks a random value $p_0 \leftarrow\!\$\ \mathbb{Z}_q^*$ and samples ciphertexts c_r and $c'_{i,0}, c'_{i,1}$ randomly from the space of valid ciphertexts (i.e., from $\{0,1\}^{\ell_m}$). It computes the remaining values together with value a and its sharing honestly similar to the ciphertexts $c_{i,0}, c_{i,1}$ and random values $b_{i,0}, b_{i,1}$. Finally, it returns the dormant one-time key dmk as an answer to this query. Using the same notation as above, $\mathcal{R}_3$ keeps a list of the form (pk_D, i, h_i), where the adversary makes a query for $\mathsf{H}_2((\mathsf{pk}_D)^{\mathsf{sk}_{i,h_i}})$. In other words, the reductions using the queries to the random oracle H_2 made by $\mathcal{A}$ in nUKE.Shared to extract $h = h_1 \ldots h_{\ell_m} = \mathsf{H}(m)$. Once known, the reduction queries $\mathsf{H}(m)$ to its signing oracle, receiving an ECDSA signature $\sigma = (r, s)$. It is worth noting that the value $\mathsf{H}(m)$ will be known to $\mathcal{R}_3$ before the adversary learns the value a, thus allowing us to encode the message σ into the ciphertexts $c_{i,0}, c_{i,1}$. The reduction first sets $\mathsf{H}_2(a) := c_r \oplus r$ and then for each $i \in [\ell_m - 1]$ it sets $\mathsf{H}_2(a||b_{i,h_i}) := u_i \oplus c'_{i,h_i}$, for some randomly sampled u_i and

$$\mathsf{H}_2(a||b_{\ell_m,h_{\ell_m}}) := (s - p_0 \sum_{i \in [\ell_m - 1]} u_i) \oplus c'_{\ell_m,h_{\ell_m}}.$$

In other words, the adversary will receive the signature (r, s) after applying the computations from DSign. Finally, it will output $l + 1$ $((m_1, \sigma_1), \ldots, (m_{l+1}, \sigma_{l+1}))$ valid message-signature pair, where for all messages they were never queried to the O_S oracle. Since we used the ECDSA signing oracle to simulate O_D, we cannot use most returned pairs as a valid ECDSA forgery. Fortunately, the reduction only used that oracle l times, while the adversary returned $l + 1$ good message-signature pair. $\mathcal{R}_3$ picks the pair (m^*, σ^*) for which it never asked $\mathsf{H}(m^*)$ to its signing oracle and returns this as a valid forgery against the unforgeability of the ECDSA signature scheme.

□

Proof (of Theorem 5.5). We will show this theorem via a series of small changes we make to experiment.

Game G_0: The original verifiability experiment $\Pr[G_0(\mathcal{A}) = 1] := \mathbf{Adv}_{\mathsf{VER}}(\mathcal{A})$.

Game G_1: The experiment aborts the game in the case that $\mathsf{osk}_S = \perp$.

Claim: We claim that the following holds:

$$|\Pr[G_1(\mathcal{A}) = 1] - \Pr[G_0(\mathcal{A}) = 1]| \leq \mathbf{Adv}_{\mathsf{Sound}}(\mathcal{R}_1).$$

Proof. We prove the claim by constructing a reduction algorithm $\mathcal{R}_1$ against the soundness Sound of NIZK scheme. Since we have that $\mathsf{osk}_S = \perp$, it must be the case that RDelegate using the provided activated osk_D from dmk_D does not give out a correct Waters decryption key. However, the statement checked by VerifyD using dmk_D ensure that the circuit is well-formed (RDelegate will always give out a correct decryption key). Clearly, the adversary had proven a false statement. Thus, when the reduction return $(\pi_{\mathsf{NIZK}}, (\mathsf{dmk}_D, \mathsf{tag}_D, \mathsf{pk}, \mathsf{pk}_D))$ to the challenger where π_{NIZK} is parsed from dmk, it will break the soundness property of the proof system. □

Game G_2: The experiment aborts the game in the case that $\mathsf{DS.Verify}(\mathsf{vk}, \mathsf{DSign}(\mathsf{osk}_S, \mathsf{sk}_s, \mathsf{m}), \mathsf{m}) = 0$.

Claim: We claim that the following holds:

$$|\Pr[G_2(\mathcal{A}) = 1] - \Pr[G_1(\mathcal{A}) = 1]| \leq \mathbf{Adv}_{\mathsf{Sound}}(\mathcal{R}_2).$$

Proof. We prove the claim by constructing a similar reduction algorithm $\mathcal{R}_2$ against the soundness Sound of NIZK scheme. Since we have that $\mathsf{DS.Verify}(\mathsf{vk}, \mathsf{DSign}(\mathsf{osk}_S, \mathsf{sk}_s, \mathsf{m}), \mathsf{m}) = 0$, it must be the case that DSign using the provided activated osk_S from dmk_S does not give out a correct ECDSA signature. However, the statement checked by VerifyD using dmk_S ensure that the circuit is well-formed (Computing DSign on osk_S will always give out a correct ECDSA signature). Clearly, the adversary had proven a false statement. Thus, when the reduction return $(\pi_{\mathsf{NIZK}}, (\mathsf{dmk}_S, \mathsf{tag}_S, \mathsf{pk}, \mathsf{pk}_S))$ to the challenger where π_{NIZK} is parsed from dmk, it will break the soundness property of the proof system. □

The adversary has no chance at winning the game anymore because the experiment always return 0.

□